МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования
Московский технический университет связи и информатики

Учебное пособие
«Введение в информационные технологии»

для студентов направлений
09.03.04 «Программная инженерия»

Москва 2026

Оглавление

ОБЩИЕ ТРЕБОВАНИЯ К ВЫПОЛНЕНИЮ И ОФОРМЛЕНИЮ ЛАБОРАТОРНЫХ РАБОТ 3

ЛАБОРАТОРНАЯ РАБОТА №1: ВВЕДЕНИЕ В СИСТЕМУ КОНТРОЛЯ ВЕРСИЙ GIT 4

ЛАБОРАТОРНАЯ РАБОТА №2: ВВЕДЕНИЕ В ЯЗЫК ПРОГРАММИРОВАНИЯ PYTHON 18

ЛАБОРАТОРНАЯ РАБОТА №3: ФУНКЦИИ В PYTHON И БАЗОВЫЕ АЛГОРИТМЫ 39

ЛАБОРАТОРНАЯ РАБОТА №4: ОРГАНИЗАЦИЯ PYTHON-ПРОЕКТА . 54

ЛАБОРАТОРНАЯ РАБОТА №5: ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ В PYTHON 69

ЛАБОРАТОРНАЯ РАБОТА №6: РАЗРАБОТКА GUI-ПРИЛОЖЕНИЙ НА PYQT6 83

ЛАБОРАТОРНАЯ РАБОТА №7: КАЧЕСТВО И НАДЁЖНОСТЬ PYTHON-КОДА: СТИЛЬ, СТАТИЧЕСКИЙ АНАЛИЗ, ОТЛАДКА И ТЕСТИРОВАНИЕ 97

ЛАБОРАТОРНАЯ РАБОТА №8: ОСНОВЫ ОПЕРАЦИОННЫХ СИСТЕМ LINUX 113

Общие требования к выполнению и оформлению лабораторных работ

Все лабораторные работы выполняются студентом самостоятельно в соответствии с заданием и оформляются в виде отчёта.

Отчёт по каждой лабораторной работе должен быть подготовлен в соответствии с установленным шаблоном оформления.

В отчёте необходимо последовательно отразить весь ход выполнения лабораторной работы.

Каждый выполненный шаг должен сопровождаться:

- кратким описанием выполненного действия;
- используемой командой, фрагментом кода или настройкой;
- рисунком, содержащим результат выполнения данного шага.

В качестве рисунков используются скриншоты, подтверждающие фактическое выполнение задания.

Скриншот должен позволять однозначно определить результат выполненного действия. На изображении должны быть видны необходимые элементы интерфейса, командной строки, исходного кода или полученного результата.

Не допускается использование скриншотов, не относящихся непосредственно к описываемому этапу работы.

Каждый рисунок должен иметь номер и подпись в соответствии с шаблоном отчёта.

Контрольные вопросы предназначены для самопроверки и подготовки к защите лабораторной работы. Преподаватель может использовать их при устном опросе. Письменные ответы на все вопросы в отчёте не требуются, если иное не указано преподавателем.

Лабораторная работа №1: Введение в систему контроля версий Git

Цель работы: познакомиться с принципами работы системы контроля версий Git, понять назначение систем контроля версий и основные понятия Git, научиться создавать локальный репозиторий и выполнять основные операции с ним через командную строку.

В ходе выполнения лабораторной работы необходимо изучить понятия рабочего каталога, индекса, коммита, истории изменений и ветки, а также научиться создавать коммиты, просматривать историю проекта, сравнивать версии файлов, работать с ветками и выполнять отмену внесённых изменений.

ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1. Что такое система контроля версий

При разработке программного обеспечения исходный код постоянно изменяется. Разработчики добавляют новые функции, исправляют ошибки, изменяют структуру проекта и экспериментируют с различными вариантами реализации.

Наиболее простой способ хранения разных состояний проекта — создание нескольких копий каталога или файла:

```
Диплом.docx
Диплом_финал.docx
Диплом_исправленный.docx
Диплом_печать.docx
Диплом_печать_исправленный.docx
Последний_Диплом_исправленный_финал.docx
```

Такой подход быстро становится неудобным. Становится сложно определить:

- какая версия является актуальной;

- какие изменения были сделаны между версиями;
- кто и когда изменил конкретный файл;
- как вернуть проект к предыдущему рабочему состоянию;
- как одновременно разрабатывать несколько вариантов проекта.

Для решения этих задач используются системы контроля версий.

Система контроля версий (Version Control System, VCS) — это программное обеспечение, предназначенное для хранения истории изменений файлов и управления различными версиями проекта.

Git является распределённой системой контроля версий. Это означает, что полноценная история репозитория может храниться локально на компьютере разработчика, а основные операции с историей не требуют постоянного подключения к серверу.

Git позволяет:

- сохранять историю изменений проекта;
- сравнивать различные состояния файлов;
- возвращаться к предыдущим версиям;
- создавать независимые направления разработки;
- объединять изменения;
- организовывать совместную работу над одним проектом.

2. Git и GitHub

Важно понимать, что **Git** и **GitHub** это не одно и то же,

Git — система контроля версий, которая устанавливается на компьютер пользователя и может работать локально, без подключения к Интернету.

GitHub — интернет-сервис для хранения Git-репозиториев и организации совместной разработки.

Помимо GitHub существуют другие сервисы для размещения Git-репозиториев, например GitLab, GitFlic и Bitbucket.

3. Основные понятия Git

3.1. Репозиторий

Репозиторий — каталог проекта, история изменений которого отслеживается Git.

Внутри Git-репозитория находится служебный каталог .git. Он содержит историю проекта, ссылки на ветки, настройки репозитория и другую служебную информацию Git.

Создание нового репозитория выполняется командой:

```
git init
```

После выполнения команды текущий каталог становится Git-репозиторием.

3.2. Рабочий каталог

Рабочий каталог содержит файлы проекта, с которыми непосредственно работает пользователь. После изменения файла Git обнаружит изменение, однако оно ещё не становится частью истории проекта.

```
project/
    main.py
    README.md
```

3.3. Индекс

Перед созданием коммита необходимо выбрать изменения, которые должны попасть в него. Для этого используется индекс Git (staging area).

Добавление одного файла в индекс:

```
git add main.py
```

Добавление всех изменений текущего каталога:

```
git add .
```

Команда git add не создаёт новую версию проекта. Она определяет, какие изменения попадут в следующий коммит.

```
Рабочий каталог
      | git add
      v
```

```
    Индекс
      |
      |git commit
      v
История репозитория
```

3.4. Коммит

Коммит — зафиксированное состояние изменений проекта в истории Git.

Для создания коммита используется команда:

```
git commit -m "Добавлен первый файл"
```

Каждый коммит содержит информацию об изменениях, авторе, дате и времени, сообщении коммита и имеет уникальный идентификатор (хеш).

Коммит следует рассматривать не просто как сохранение файла, а как логически завершённое изменение проекта. Хорошее сообщение коммита должно кратко объяснять, что было изменено.

```
Добавлена проверка пользовательского ввода
Исправлена ошибка расчёта среднего значения
```

Не рекомендуется использовать неинформативные сообщения по типу:

```
commit1
изменения
готово
123
```

4. Проверка состояния репозитория

Одной из наиболее часто используемых команд Git является:

```
git status
```

Команда показывает, какие файлы были изменены, какие файлы ещё не отслеживаются Git, какие изменения добавлены в индекс и в какой ветке находится пользователь.

На начальном этапе изучения Git мы рекомендуем выполнять git status после большинства операций и анализировать, как меняется состояние репозитория.

5. История изменений

Для просмотра полной истории коммитов используется команда:

```
git log
```

Более компактный вариант:

```
git log --oneline
```

Пример истории:

```
83f28a1 Добавлен расчёт среднего значения
7bc214f Добавлена обработка пользовательского ввода
54af927 Создана первая версия программы
```

Историю Git можно представить как последовательность связанных коммитов:

```
A ---- B ---- C ---- D
```

Каждый новый коммит создаётся на основе предыдущего состояния истории.

6. Просмотр изменений

Для просмотра изменений рабочего каталога, которые ещё не были добавлены в индекс, используется:

```
git diff
```

Для просмотра изменений, уже добавленных в индекс:

```
git diff --staged
```

Перед созданием коммита полезно проверить, какие именно изменения будут сохранены в истории.

7. Что такое ветка

Во время разработки часто необходимо внести изменения или реализовать новую функциональность, не нарушая текущую рабочую версию проекта. Для этого Git использует ветки.

Ветка — отдельная линия развития проекта, представленная указателем на определённый коммит.

```
A ---- B ---- C  main
               \
                D ---- E  feature
```

Ветки позволяют параллельно разрабатывать новую функциональность, исправлять ошибки и проводить эксперименты. Основная ветка современного репозитория обычно называется main.

Просмотр существующих веток:

```
git branch
```

Создание новой ветки:

```
git branch feature
```

Переход в ветку:

```
git switch feature
```

Создать новую ветку и сразу перейти в неё:

```
git switch -c feature
```

В старых руководствах для создания и переключения между ветками часто используется команда `git checkout`. Начиная с Git 2.23 для этих операций рекомендуется использовать специализированную команду `git switch`.

8. Объединение веток

После завершения работы в отдельной ветке её изменения можно объединить с основной веткой.

```
git switch main
git merge feature
```

Если изменения не противоречат друг другу, Git выполнит объединение автоматически.

9. Конфликты слияния

Если две ветки изменили одну и ту же часть файла по-разному, Git не всегда может самостоятельно определить правильный результат. В этом случае возникает конфликт слияния (merge conflict).

```
<<<<<<< HEAD
print("Hello")
=======
print("Hello, world!")
>>>>>>> feature
```

Разработчик должен вручную выбрать правильный вариант, удалить служебные маркеры конфликта, проверить файл, добавить исправленную версию в индекс и завершить слияние коммитом.

Конфликт не является ошибкой Git. Git хранит обе версии и требует от разработчика принять решение, потому что только человек понимает смысл изменений.

10. Отмена изменений

Фраза «отмена изменений» в Git может означать несколько разных действий. Перед выполнением отмены необходимо определить, на каком этапе находятся изменения: только в рабочем каталоге, уже в индексе или уже сохранены в коммите.

10.1. Отмена незакоммиченных изменений

Если файл был изменён, но необходимо вернуть его состояние из последнего коммита:

```
git restore filename
```

Несохранённые изменения этого файла при этом будут потеряны.

10.2. Удаление файла из индекса

Если изменения были добавлены с помощью git add, но их пока не следует включать в коммит:

```
git restore --staged filename
```

Изменения останутся в рабочем каталоге, но будут удалены из индекса.

10.3. Отмена уже созданного коммита

Для безопасной отмены изменений уже существующего коммита используется команда git revert:

```
git revert <commit>
```

Git создаст новый коммит, который отменит изменения указанного коммита. Исходный коммит при этом останется в истории.

```
До:     A ---- B ---- C
После:   A ---- B ---- C ---- D
                              ^
                              D отменяет изменения C
```

10.4. Изменение истории

Git также имеет команду git reset, которая способна перемещать указатель ветки на другой коммит и тем самым изменять историю. Один из наиболее опасных вариантов:

```
git reset --hard HEAD~1
```

Эта команда может удалить незакоммиченные изменения и изменить историю ветки. В рамках данной лабораторной работы использовать git reset --hard не требуется.

Главное различие: git revert создаёт новый коммит, отменяющий старый, а git reset может изменить уже существующую историю.

Изменение истории особенно опасно, если соответствующие коммиты уже были опубликованы и используются другими разработчиками. Поэтому перед «откатом» необходимо понимать, что именно требуется отменить и кто ещё работает с данной историей.

11. Установка и первоначальная настройка Git

После установки Git проверьте его доступность в командной строке:

```
git --version
```

Перед созданием первых коммитов необходимо указать имя пользователя и электронную почту:

```
git config --global user.name "Иван Иванов"
git config --global user.email "student@example.com"
```

Проверить текущие настройки можно командой:

```
git config --list
```

ПРАКТИЧЕСКАЯ ЧАСТЬ

Задание 1: Создание локального репозитория

1. Создайте каталог git_lab и перейдите в него через командную строку.
2. Создайте новый локальный Git-репозиторий командой git init.
3. Выполните git status и изучите полученный результат.

Задание 2: Первый файл и первый коммит

1. Создайте файл notes.txt.
2. Добавьте в него строку «Первая версия файла».
3. Выполните git status.
4. Добавьте файл в индекс командой git add notes.txt.
5. Снова выполните git status и сравните результат с предыдущим.
6. Создайте первый коммит с сообщением «Добавлен файл notes.txt».
7. Просмотрите историю командами git log и git log --oneline.

Задание 3: Изменение файла

1. Добавьте в notes.txt ещё две строки.
2. Выполните git status.
3. Просмотрите внесённые изменения командой git diff.
4. Добавьте файл в индекс.

5. Просмотрите подготовленные к коммиту изменения командой git diff --staged.

6. Создайте второй коммит. Сообщение коммита придумайте самостоятельно так, чтобы оно отражало содержание изменений.

Задание 4: Создание истории проекта

1. Внесите ещё минимум три последовательных изменения в notes.txt.

2. Каждое логически завершённое изменение оформите отдельным коммитом.

3. Добейтесь того, чтобы история репозитория содержала минимум пять коммитов.

4. Просмотрите итоговую историю командой git log --oneline.

Задание 5: Работа с веткой

1. Создайте новую ветку experiment и перейдите в неё.

2. Создайте файл experiment.txt и добавьте в него произвольный текст.

3. Добавьте файл в индекс и создайте коммит.

4. Просмотрите историю.

5. Вернитесь в ветку main.

6. Проверьте содержимое каталога и объясните, почему файл experiment.txt отсутствует в ветке main.

Задание 6: Объединение веток

1. Находясь в ветке main, выполните слияние с веткой experiment.

2. Проверьте наличие файла experiment.txt.

3. Просмотрите историю проекта после слияния.

Задание 7: Отмена незакоммиченного изменения

1. Измените содержимое notes.txt.

2. Просмотрите изменения командой git diff.

3. Не создавая коммит, верните файл к предыдущему состоянию командой git restore notes.txt.

4. Убедитесь, что изменение было отменено.

Задание 8: Работа с индексом

1. Измените notes.txt.
2. Добавьте изменение в индекс командой git add notes.txt.
3. Проверьте состояние репозитория.
4. Удалите изменение из индекса командой git restore --staged notes.txt.
5. Снова выполните git status.
6. Ответьте на вопрос: было ли удалено само изменение файла?

Задание 9: Отмена коммита

1. Внесите новое изменение в notes.txt и создайте коммит с сообщением «Экспериментальное изменение».
2. Найдите идентификатор созданного коммита с помощью git log --oneline.
3. Отмените его командой git revert <идентификатор_коммита>.
4. Просмотрите историю.
5. Ответьте на вопрос: исчез ли отменённый коммит из истории?

Задание 10: Самостоятельная работа

Создайте новую ветку с произвольным названием. Необходимо самостоятельно подобрать команды Git и выполнить следующие действия:

1. создать новый текстовый файл;
2. создать первый коммит;
3. изменить файл;
4. создать второй коммит;
5. просмотреть различия между версиями;
6. вернуться в ветку main;
7. объединить созданную ветку с main;
8. вывести итоговую историю репозитория.

Требования к результату работы

После выполнения лабораторной работы студент должен уметь объяснить назначение и принцип действия следующих команд:

```
git init
git status
git add
git commit
git log
git diff
git branch
git switch
git merge
git restore
git revert
```

Студент также должен понимать разницу между Git и GitHub, рабочим каталогом и индексом, изменением файла и коммитом, коммитом и веткой, а также между отменой изменений и изменением истории Git.

Контрольные вопросы для самопроверки

1. Что такое система контроля версий и для чего она используется?
2. Для чего используется Git?
3. Чем Git отличается от GitHub?
4. Что такое Git-репозиторий?
5. Для чего используется каталог .git?
6. Что такое рабочий каталог?
7. Что такое индекс Git?
8. Что делает команда git add?
9. Что такое коммит?
10. Почему рекомендуется создавать небольшие логически завершённые коммиты?
11. Что показывает команда git status?
12. Для чего используется git diff?
13. Что представляет собой ветка в Git?
14. Для чего используются ветки?
15. Что происходит при выполнении git merge?
16. Что такое конфликт слияния?
17. Почему Git не всегда может автоматически разрешить конфликт?
18. В чём разница между git restore и git revert?
19. Почему отмена изменений в Git может быть сложной операцией?
20. В чём принципиальная разница между git revert и git reset?
21. Почему изменение уже опубликованной истории может создать проблемы при командной разработке?
22. Может ли Git работать без GitHub и подключения к Интернету?

Лабораторная работа №2: Введение в язык программирования Python

Цель работы: познакомиться с языком программирования Python, принципами выполнения программ, стандартной библиотекой и базовым синтаксисом языка; установить и настроить интерпретатор Python и среду разработки; получить практические навыки ввода и вывода данных, использования переменных, основных типов данных, арифметических операций и циклических конструкций.

В ходе выполнения лабораторной работы необходимо научиться создавать и запускать программы на Python, использовать переменные и основные типы данных, получать данные от пользователя, выполнять преобразование типов и арифметические операции, а также организовывать повторяющиеся действия с помощью циклов for и while.

ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1. Язык программирования Python

Python — высокоуровневый язык программирования общего назначения. Он используется при разработке прикладных и серверных программ, автоматизации, обработке и анализе данных, научных вычислениях, машинном обучении, компьютерном зрении и во многих других областях.

Одной из особенностей Python является сравнительно простой и читаемый синтаксис. Благодаря этому язык широко применяется как для обучения программированию, так и при разработке реальных программных систем.

Простейшая программа на Python может состоять всего из одной инструкции:

```
print("Hello, World!")
```

После выполнения программы на экран будет выведено:

```
Hello, World!
```

2. Интерпретатор Python и исходный код

Исходный код программы Python обычно хранится в текстовых файлах с расширением .py. Например:

```
main.py
```

Для выполнения программы необходим интерпретатор Python — программа, которая обрабатывает и выполняет инструкции исходного кода.

Важно различать исходный код, интерпретатор и среду разработки. Исходный код содержит программу, интерпретатор выполняет её, а среда разработки или редактор используется для удобного написания, запуска и отладки кода.

Важно. Интерпретатор и компилятор

При изучении языков программирования часто выделяют два основных способа выполнения программ: компиляцию и интерпретацию.

Компилятор предварительно преобразует исходный код программы в другую форму, чаще всего в машинный код или исполняемый файл, который затем может быть запущен операционной системой.

Упрощённо этот процесс можно представить следующим образом:

```
Исходный код
      ↓
      Компилятор
            ↓
      Исполняемый файл
            ↓
      Выполнение
```

Такой подход характерен, например, для языков C и C++.

Интерпретатор выполняет программу с помощью специальной программы-интерпретатора. Пользователь запускает исходный файл через интерпретатор, после чего тот обрабатывает инструкции программы и обеспечивает их выполнение.

Python обычно относят к интерпретируемым языкам, поскольку для запуска программы пользователь, как правило, использует интерпретатор Python:

```
python main.py
```

Однако важно понимать, что реальный процесс выполнения Python сложнее этой упрощённой схемы.

В наиболее распространённой реализации Python → **CPython** → исходный код сначала преобразуется во внутреннее промежуточное представление, называемое **байт-кодом**, после чего этот байт-код выполняется виртуальной машиной Python.

```
Исходный код Python (.py)
↓
CPython
↓
Байт-код
↓
Виртуальная машина Python
↓
Выполнение
```

Поэтому противопоставление «компилируемый язык» и «интерпретируемый язык» является некоторым упрощением. Во многих современных языках при выполнении программы могут одновременно использоваться несколько этапов преобразования и интерпретации.

3. Установка Python

Для выполнения лабораторных работ необходимо установить актуальную версию Python 3. Рекомендуется использовать официальный дистрибутив с сайта Python:

```
https://www.python.org/downloads/
```

На странице загрузки выберите установщик для своей операционной системы. В Windows при использовании классического установщика рекомендуется включить опцию добавления Python в PATH, если она

предлагается установщиком. Это позволит запускать интерпретатор из командной строки.

После установки откройте PowerShell, Windows Terminal или командную строку и выполните:

```
python --version
```

В некоторых конфигурациях Windows для запуска Python используется команда:

```
py --version
```

Если установка выполнена правильно, в терминале будет отображена установленная версия Python.

4. Установка и настройка среды разработки

Для выполнения лабораторных работ рекомендуется использовать Visual Studio Code. Скачать редактор можно с официального сайта:

```
https://code.visualstudio.com/
```

Visual Studio Code является редактором исходного кода и сам по себе не заменяет интерпретатор Python. Для работы необходимы установленный Python, Visual Studio Code и расширение Python для Visual Studio Code.

После установки Visual Studio Code откройте раздел Extensions, найдите расширение Python от Microsoft и установите его.

Затем выберите интерпретатор Python. Для этого нажмите Ctrl + Shift + P, введите команду:

```
Python: Select Interpreter
```

и выберите установленную версию Python. После этого Visual Studio Code будет использовать выбранный интерпретатор при запуске программ.

5. Создание и запуск программы

Создайте отдельную рабочую директорию для проекта и откройте её в Visual Studio Code. Внутри директории создайте файл main.py.

Добавьте в него следующий код:

```
print("Hello, World!")
```

Программу можно запустить из Visual Studio Code или через встроенный терминал. Для запуска из командной строки используется:

```
python main.py
```

или, если в Windows используется Python Launcher:

```
py main.py
```

6. Базовый синтаксис Python

Программа Python состоит из инструкций. Обычно каждая инструкция записывается на отдельной строке:

```
print("Первая строка")
print("Вторая строка")
```

Python чувствителен к регистру. Имена **name**, **Name** и **NAME** считаются разными. Регистр также имеет значение для встроенных функций и ключевых слов языка.

Комментарии используются для пояснения исходного кода и не выполняются интерпретатором. Однострочный комментарий начинается с символа #:

```
# Вывод сообщения на экран
print("Hello")
```

7. Переменные

Переменная используется для хранения значения. В Python переменная создаётся в момент присваивания ей значения:

```
age = 18
```

В данном выражении «age» это имя переменной, знак «=» — оператор присваивания, а «18» — присваиваемое значение.

Пример нескольких переменных:

```
name = "Иван"
age = 18
height = 1.82
```

Значение переменной можно изменить во время выполнения программы:

```
score = 10
print(score)
score = 25
print(score)
```

Для имён переменных следует выбирать понятные названия, отражающие назначение хранимых данных. Например: student_age, product_price, user_name.

8. Основные типы данных

Каждое значение в Python имеет определённый тип. В данной лабораторной работе используются четыре основных типа данных.

8.1. Целые числа — int

```
age = 18
temperature = -5
count = 100
```

8.2. Вещественные числа — float

```
height = 1.82
price = 199.99
temperature = -3.5
```

Для записи дробной части числа используется точка, а не запятая.

8.3. Строки — str

```
name = "Иван"
city = "Москва"
```

Строковые значения заключаются в одинарные или двойные кавычки.

8.4. Логический тип — bool

```
is_student = True
is_admin = False
```

Логический тип имеет два значения: True и False. Они записываются с заглавной буквы.

8.5. Определение типа значения

Для определения типа объекта используется встроенная функция type():

```
age = 18
print(type(age))
```

9. Вывод данных

Для вывода информации на экран используется функция print(). В неё можно передавать как текст, так и значения переменных:

```
name = "Иван"
age = 18
print("Имя:", name)
print("Возраст:", age)
```

Функция print() также может принимать несколько значений одновременно:

```
print("Имя:", name, "Возраст:", age)
```

10. Ввод данных

Для получения данных от пользователя используется функция input(). При её выполнении программа ожидает ввод пользователя с клавиатуры:

```
name = input("Введите имя: ")
print("Здравствуйте,", name)
```

Важно учитывать, что функция input() всегда возвращает значение типа str. Даже если пользователь вводит число, без дополнительного преобразования оно будет храниться как строка.

11. Преобразование типов

Для явного преобразования значений между типами используются функции int(), float(), str() и bool(). Например:

```
age = int(input("Введите возраст: "))
height = float(input("Введите рост: "))
```

Сначала input() получает строку, затем int() или float() преобразует её в соответствующий числовой тип.

12. Арифметические операции

Python поддерживает основные арифметические операции:

+ — сложение;

- — вычитание;

* — умножение;

/ — обычное деление;

// — целочисленное деление;

% — остаток от деления;

** — возведение в степень.

Пример:

```
a = 10
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

Как и в математике, операции имеют различный приоритет. Для явного изменения порядка вычислений используются круглые скобки:

```
result = (2 + 3) * 4
```

12.1. Операции сравнения

Для проверки отношений между значениями в Python используются операции сравнения. Результатом такой операции является логическое значение True или False.

== — равно;

!= — не равно;

> — больше;

< — меньше;

>= — больше или равно;

<= — меньше или равно.

Например:

```
a = 10
print(a > 5)
print(a == 10)
print(a <= 3)
```

Операции сравнения особенно важны при работе с циклом while, поскольку его выполнение продолжается до тех пор, пока заданное логическое условие остаётся истинным.

13. Логические конструкции и условный оператор

До этого момента инструкции программы выполнялись последовательно, сверху вниз. Однако во многих программах необходимо выполнять различные действия в зависимости от определённого условия.

Например:

- если возраст пользователя не меньше 18 лет, то разрешить доступ;
- если температура ниже нуля, то вывести предупреждение;
- если введённое число положительное, то выполнить определённое действие.

Для изменения хода выполнения программы в Python используется условный оператор if.

13.1. Условный оператор if

Общий вид условного оператора:

```
if условие:
    действие
```

Пример:

```
age = 20
if age >= 18:
    print("Пользователь совершеннолетний")
```

Сначала Python вычисляет логическое выражение age >= 18. Если его результат равен True, выполняется блок кода, расположенный внутри конструкции if. Если результат равен False, данный блок пропускается.

13.2. Конструкция if-else

Если необходимо выполнить одно действие при истинном условии и другое при ложном, используется конструкция if-else.

```
age = int(input("Введите возраст: "))

if age >= 18:
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

Если условие age >= 18 равно True, выполняется первый блок. В противном случае выполняется блок else.

13.3. Конструкция if-elif-else

Если необходимо проверить несколько взаимоисключающих условий, используется конструкция elif.

```
temperature = float(input("Введите температуру: "))

if temperature > 0:
    print("Температура выше нуля")
elif temperature < 0:
    print("Температура ниже нуля")
else:
    print("Температура равна нулю")
```

Проверка выполняется сверху вниз. Как только найдено первое истинное условие, соответствующий блок выполняется, а остальные ветви конструкции пропускаются.

13.4. Логические операторы

Иногда одного условия недостаточно. Для объединения нескольких логических выражений используются операторы and, or и not.

Оператор and

Выражение с оператором and истинно только в том случае, если истинны оба условия.

```
age = 20
has_ticket = True

if age >= 18 and has_ticket:
    print("Доступ разрешён")
```

В данном примере одновременно должны выполняться два условия: возраст пользователя не меньше 18 лет и билет имеется.

Оператор or

Выражение с оператором or истинно, если истинно хотя бы одно из условий.

```
is_admin = False
is_moderator = True

if is_admin or is_moderator:
    print("Доступ к панели управления разрешён")
```

Оператор not

Оператор not изменяет логическое значение на противоположное.

```
is_blocked = False

if not is_blocked:
    print("Пользователь не заблокирован")
not True  -> False
not False -> True
```

Объединение нескольких условий

Логические операторы позволяют создавать более сложные условия. Например:

```
age = int(input("Введите возраст: "))
has_ticket = input("Есть билет? ").lower() == "да"

if age >= 18 and has_ticket:
    print("Проход разрешён")
else:
    print("Проход запрещён")
```

При использовании сложных логических выражений допускается применять круглые скобки, если они делают условие более понятным.

```
if (age >= 18) and has_ticket:
    print("Доступ разрешён")
```

Сравнивать логическую переменную с True обычно не требуется. Если переменная уже имеет тип bool, её можно использовать непосредственно в условии.

14. Циклические конструкции

Во многих программах одни и те же действия необходимо выполнять несколько раз. Для организации повторяющихся действий используются циклы.

В Python основными циклическими конструкциями являются for и while.

14.1. Цикл for

Цикл for используется для последовательного перебора элементов или повторения действий заданное количество раз. На начальном этапе часто используется совместно с функцией range().

```
for i in range(5):
    print(i)
```

В результате будут выведены числа от 0 до 4. Значение 5 в данном случае не входит в последовательность.

Функция range() может принимать начальное и конечное значения:

```
for i in range(1, 6):
    print(i)
```

В этом случае будут выведены числа от 1 до 5.

Также можно указать шаг изменения:

```
for i in range(0, 11, 2):
    print(i)
```

В результате будут выведены чётные числа от 0 до 10.

14.2. Отступы в Python

В Python отступы являются частью синтаксиса и определяют, какие инструкции относятся к телу цикла и другим составным конструкциям.

```
for i in range(3):
    print("Внутри цикла", i)

print("После цикла")
```

Инструкция с отступом выполняется на каждой итерации, а инструкция без отступа выполняется после завершения цикла. Рекомендуется использовать одно нажатие клавиши TAB (четыре пробела) для одного уровня отступа.

14.3. Цикл while

Цикл while выполняет блок инструкций до тех пор, пока заданное условие остаётся истинным.

```
count = 1
while count <= 5:
    print(count)
    count = count + 1
```

Перед каждой итерацией проверяется условие count <= 5. Когда оно становится ложным, выполнение цикла прекращается.

При использовании while важно изменять переменные, участвующие в условии завершения. Если условие никогда не станет ложным, цикл может выполняться бесконечно.

14.4. Накопление результата в цикле

Циклы часто используются для последовательного вычисления общего результата. Например, сумму чисел от 1 до 5 можно вычислить следующим образом:

```
total = 0
for i in range(1, 6):
    total = total + i

print(total)
```

На каждой итерации очередное значение i прибавляется к переменной total.

15. Стандартная библиотека Python

Вместе с Python устанавливается стандартная библиотека — набор готовых модулей, предоставляющих дополнительные возможности. В данной лабораторной работе модуль math используется только как простой пример обращения к стандартной библиотеке.

Для подключения модуля используется инструкция import:

```
import math
```

После подключения можно обращаться к объектам модуля, например к числу π и функции вычисления квадратного корня:

```
import math
print(math.pi)
print(math.sqrt(25))
```

Подробное устройство модулей и пакетов, различные способы импорта, создание собственных модулей и организация программ в пакеты будут рассмотрены в отдельной лабораторной работе. На данном этапе необходимо только понимать, что Python содержит стандартную библиотеку и её возможности можно подключать с помощью import.

ПРАКТИЧЕСКАЯ ЧАСТЬ

Задание 1: Установка и проверка Python

1. Скачайте и установите актуальную версию Python 3 с официального сайта.
2. Откройте командную строку или PowerShell.
3. Проверьте установленную версию командой python --version или py --version.
4. Зафиксируйте результат выполнения команды в отчёте.

Задание 2: Установка и настройка Visual Studio Code

1. Установите Visual Studio Code.
2. Установите расширение Python от Microsoft.
3. Выберите установленный интерпретатор с помощью команды Python: Select Interpreter.
4. Создайте рабочую директорию python_lab_02 и откройте её в Visual Studio Code.
5. Зафиксируйте выбранный интерпретатор в отчёте.

Задание 3: Первая программа

1. Создайте файл main.py.
2. Добавьте команду print("Hello, World!").
3. Запустите программу из Visual Studio Code.
4. Запустите ту же программу через терминал командой python main.py или py main.py.
5. Зафиксируйте оба результата запуска.

Задание 4: Переменные, типы данных и вывод

1. Создайте переменные name, age, height и is_student, содержащие соответственно имя, возраст, рост и логическое значение, указывающее, является ли пользователь студентом.
2. Выведите значения всех переменных.
3. С помощью функции type() выведите тип каждой переменной.

4. Измените хотя бы два значения переменных и повторно выведите результат.

Задание 5: Ввод данных и преобразование типов

1. Создайте программу, запрашивающую имя, фамилию и возраст пользователя.

2. Сначала сохраните возраст без преобразования и определите его тип с помощью type().

3. Измените программу так, чтобы возраст преобразовывался в int.

4. Добавьте ввод роста пользователя и преобразуйте его в float.

5. Выведите введённые данные и типы числовых переменных.

Задание 6: Арифметические операции

1. Создайте две числовые переменные a = 15 и b = 4.

2. Вычислите и выведите сумму, разность, произведение, результат обычного и целочисленного деления, остаток от деления и результат возведения a в степень b.

3. Сравните результаты операций / и // и объясните различие в отчёте.

4. Измените выражение 2 + 3 * 4 с помощью скобок так, чтобы результат вычисления изменился.

Задание 7: Простой калькулятор

1. Запросите у пользователя два числа типа float.

2. Вычислите их сумму, разность, произведение и результат деления.

3. Выведите результаты в понятном виде.

4. Проверьте работу программы минимум на двух различных наборах исходных данных.

Задание 8: Цикл for и функция range()

1. С помощью цикла for выведите числа от 1 до 10.

2. Измените программу так, чтобы выводились числа от 10 до 1. Используйте соответствующие параметры range().

3. С помощью range() выведите все чётные числа от 0 до 20 включительно.

4. Измените шаг последовательности и продемонстрируйте результат в отчёте.

Задание 9: Вычисления в цикле for

1. Запросите у пользователя целое положительное число n.

2. С помощью цикла for вычислите сумму всех целых чисел от 1 до n включительно.

3. Выведите полученный результат.

4. Проверьте программу для нескольких значений n.

Задание 10: Цикл while

1. Создайте переменную count со значением 10.

2. С помощью цикла while выведите числа от 10 до 1.

3. После завершения цикла выведите сообщение «Цикл завершён».

4. Объясните в отчёте, какое условие завершает цикл и почему переменная count должна изменяться внутри цикла.

Задание 11: Стандартная библиотека на примере math

1. Подключите модуль math.

2. Создайте переменную radius и задайте ей произвольное положительное значение.

3. Используя math.pi, вычислите длину окружности и площадь круга.

4. С помощью math.sqrt() вычислите квадратный корень из произвольного положительного числа.

5. Зафиксируйте результат и помните, что подробная работа с модулями и пакетами будет рассмотрена в отдельной лабораторной работе.

Задание 12: Работа с условными конструкциями

1. Напишите программу, которая запрашивает у пользователя целое число.

2. С помощью конструкции if-elif-else определите, является ли введённое число положительным, отрицательным или равным нулю.

3. Выведите соответствующее сообщение.

Пример работы программы:

```
Введите число: -7
Число отрицательное
```

4. Дополните программу проверкой чётности числа. Для определения остатка от деления используйте оператор %.

Пример:

```
Введите число: 12
Число положительное
Число чётное
```

5. Продемонстрируйте работу программы минимум для трёх различных входных значений: положительного, отрицательного и нуля.

Задание 13: Логические операторы

Напишите программу проверки возможности допуска пользователя к выполнению лабораторной работы.

1. Создайте переменную age, содержащую возраст пользователя.

2. Создайте логическую переменную has_access, определяющую наличие допуска к лабораторной работе.

3. Доступ должен разрешаться только в том случае, если пользователь достиг возраста 18 лет и имеет допуск.

4. Для объединения условий используйте логический оператор and.

Пример результата:

```
Возраст: 19
Допуск имеется: True
Доступ разрешён
```

Другой вариант:

```
Возраст: 17
Допуск имеется: True
Доступ запрещён
```

5. Измените значения входных данных и продемонстрируйте работу программы для различных комбинаций условий.

6. В отчёте объясните, почему в данной задаче используется оператор and, а не or.

Задание 14: Самостоятельная работа

Напишите программу для анализа результатов измерений. Пользователь вводит количество измерений n, после чего программа с помощью цикла последовательно запрашивает n числовых значений. Программа должна вычислить и вывести сумму всех введённых значений и их среднее арифметическое.

Требования к программе:

количество измерений должно храниться в переменной типа int;

сами измерения должны преобразовываться в тип float;

для повторного ввода значений должен использоваться цикл for или while;

сумма должна накапливаться в отдельной переменной;

после завершения цикла программа должна вывести сумму и среднее значение.

Последовательность команд для решения задания необходимо определить самостоятельно.

Требования к результату работы

После выполнения лабораторной работы студент должен уметь объяснить назначение интерпретатора Python и среды разработки, создавать и запускать программы, использовать переменные и типы int, float, str и bool, выполнять ввод и вывод данных, преобразование типов и основные арифметические операции.

Студент также должен понимать принцип работы циклов for и while, назначение функции range(), роль отступов в Python, уметь выполнять повторяющиеся действия и накапливать результат в цикле. Необходимо понимать назначение стандартной библиотеки и инструкции import на базовом уровне; подробное изучение модулей и пакетов выполняется в отдельной лабораторной работе.

Контрольные вопросы для самопроверки

1. Что представляет собой язык программирования Python?
2. Что такое интерпретатор и какую роль он выполняет?
3. Какое расширение обычно имеют файлы исходного кода Python?
4. Чем Visual Studio Code отличается от интерпретатора Python?
5. Как проверить установленную версию Python из командной строки?
6. Для чего используется функция print()?
7. Что такое переменная?
8. Какие основные типы данных были рассмотрены в лабораторной работе?
9. Для чего используется функция type()?
10. Какой тип данных возвращает функция input()?
11. Для чего используются функции int(), float() и str()?
12. Чем операции / и // отличаются друг от друга?
13. Для чего используется оператор %?
14. Как в Python выполняется возведение в степень?
15. Как с помощью скобок изменить порядок выполнения арифметических операций?
16. Какие операции сравнения используются в Python и какой тип результата они возвращают?
17. Для чего в программах используются циклы?
18. В каких случаях удобно использовать цикл for?
19. Для чего используется функция range()?
20. Какие аргументы может принимать range()?
21. Какую роль играют отступы в теле цикла Python?
22. В каких случаях используется цикл while?
23. Что может произойти, если условие цикла while никогда не станет ложным?
24. Как организовать накопление суммы значений внутри цикла?
25. Что такое стандартная библиотека Python?

26. Для чего используется инструкция import?

27. Почему модуль math в данной работе рассматривается только как пример стандартной библиотеки?

28. Для чего используется условный оператор if?

29. Чем конструкции if, if-else и if-elif-else отличаются друг от друга?

30. Какое значение должно иметь условие для выполнения блока if?

31. Какую роль выполняют отступы в Python?

32. Чем оператор = отличается от оператора ==?

33. Какие операции сравнения поддерживаются в Python?

34. Для чего используется логический оператор and?

35. Для чего используется логический оператор or?

36. Что делает оператор not?

37. Какой тип данных является результатом операции сравнения?

38. Почему выражение has_access == True обычно можно заменить на has_access

Лабораторная работа №3: Функции в Python и базовые алгоритмы

Цель работы: изучить принципы создания и использования функций в языке Python, научиться передавать данные в функции и возвращать результаты, понять области видимости переменных, освоить работу со списками и кортежами, срезами и встроенными функциями, а также получить практический опыт реализации базовых алгоритмов обработки последовательностей и применения lambda-функций.

В ходе выполнения лабораторной работы необходимо научиться разбивать программу на логически завершённые функции, использовать параметры и возвращаемые значения, обрабатывать последовательности данных и самостоятельно реализовывать простейшие алгоритмы поиска, подсчёта и агрегирования.

ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1. Функции и их назначение

По мере увеличения программы количество повторяющихся действий и связанных между собой инструкций возрастает. Если один и тот же фрагмент кода используется несколько раз, его повторное копирование усложняет сопровождение программы и повышает вероятность ошибок.

Функция — это именованный блок программного кода, который выполняет определённую задачу. Функции позволяют разбивать программу на небольшие логически законченные части и повторно использовать уже написанный код.

Простейшее определение функции:

```
def greet():
    print("Здравствуйте!")
```

Ключевое слово **def** сообщает Python о создании функции. После него указывается имя функции и круглые скобки. Тело функции отделяется отступом. Определение функции само по себе не выполняет её тело. Для выполнения функцию необходимо вызвать:

```
greet()
```

2. Параметры и аргументы функции

Функция может получать данные извне. Значения, которые функция ожидает получить, описываются через параметры.

```
def greet(name):
    print("Здравствуйте,", name)

greet("Иван")
```

В данном примере name это параметр функции, а строка "Иван" это аргумент, переданный при вызове функции.

Функция может иметь несколько параметров:

```
def add(a, b):
    print(a + b)

add(5, 3)
```

При вызове значения аргументов связываются с параметрами функции. По умолчанию это происходит по их позиции.

3. Возвращаемое значение и оператор return

Функция может не только выполнять действия, но и возвращать результат в место вызова. Для этого используется оператор return. Если функция не содержит оператора `return` с возвращаемым значением, Python автоматически возвращает значение `None`.

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)
```

Важно различать print() и return. Функция print() выводит значение на экран, а return передаёт результат вызывающему коду. Возвращённое значение можно сохранить в переменную, использовать в выражении или передать другой функции.

```
def square(number):
    return number ** 2

result = square(5) * 2
print(result)
```

После выполнения return работа функции завершается. Инструкции, расположенные в этом же блоке после выполненного return, выполнены не будут.

4. Аргументы по умолчанию и именованные аргументы

Для параметра можно определить значение по умолчанию. Такое значение будет использоваться, если соответствующий аргумент не передан при вызове.

```
def greet(name, message="Здравствуйте"):
    print(message, name)

greet("Иван")
greet("Иван", "Добрый вечер")
```

Аргументы также можно передавать по имени параметра:

```
greet(message="Добрый вечер", name="Иван")
```

Именованные аргументы делают вызов функции более понятным и позволяют изменять порядок передачи значений.

5. Области видимости переменных

Переменная доступна не во всех частях программы. Область программы, в которой можно обратиться к переменной по её имени, называется областью видимости.

Переменная, созданная внутри функции, является локальной для этой функции:

```
def example():
    value = 20
    print(value)

example()
```

Обратиться к value за пределами функции в данном примере нельзя.

Переменная, созданная вне функций, находится в глобальной области видимости:

```
value = 10

def example():
    value = 20
    print(value)

example()
print(value)
```

Программа выведет сначала 20, затем 10. Локальная переменная value внутри функции не изменяет глобальную переменную с таким же именем.

В Python существует ключевое слово global, позволяющее явно обращаться к глобальной переменной из функции:

```
count = 0

def increment():
    global count
    count += 1
```

Однако злоупотреблять глобальными переменными не рекомендуется. В большинстве случаев данные следует передавать в функцию через параметры, а результат возвращать с помощью return.

6. Списки

Список (list) — изменяемая последовательность элементов. Список позволяет хранить несколько значений в одной переменной.

```
numbers = [10, 20, 30, 40, 50]
names = ["Анна", "Иван", "Мария"]
```

Элементы списка нумеруются начиная с нуля. Для обращения к элементу используется индекс:

```
print(numbers[0])
print(numbers[2])
print(numbers[-1])
```

Отрицательные индексы позволяют обращаться к элементам с конца последовательности: индекс -1 соответствует последнему элементу.

Поскольку список является изменяемым объектом, отдельные элементы можно заменять:

```
numbers[1] = 100
```

Для добавления и удаления элементов часто используются методы:

```
numbers.append(60)
numbers.remove(30)
last_value = numbers.pop()
```

Функция len() возвращает количество элементов последовательности:

```
print(len(numbers))
```

7. Перебор элементов списка

Для последовательной обработки элементов списка удобно использовать цикл for:

```
numbers = [10, 20, 30, 40]

for number in numbers:
    print(number)
```

Если при обработке необходим индекс элемента, можно использовать range() и len():

```
for i in range(len(numbers)):
    print(i, numbers[i])
```

8. Срезы списков

Срез позволяет получить часть последовательности. Общий вид среза:

```
sequence[start:stop:step]
```

Параметр start задаёт начальный индекс, stop — конечную границу, которая не включается в результат, step — шаг выборки.

```
numbers = [10, 20, 30, 40, 50, 60]


print(numbers[1:4])
print(numbers[:3])
print(numbers[3:])
print(numbers[::2])
print(numbers[::-1])
```

Срез numbers[1:4] содержит элементы с индексами 1, 2 и 3. Значение с индексом 4 в результат не входит.

Срез всего списка может использоваться для создания поверхностной копии списка:

```
copy_numbers = numbers[:]
```

9. Кортежи

Кортеж (tuple) — последовательность элементов, похожая на список, но не допускающая изменения своего содержимого после создания. Это свойство называется неизменяемостью.

```
point = (10, 20)
colors = ("red", "green", "blue")
```

Обращение к элементам и использование срезов выполняются так же, как для списков:

```
print(point[0])
print(colors[-1])
print(colors[0:2])
```

Попытка изменить элемент кортежа приведёт к ошибке:

```
point[0] = 15  # ошибка
```

Кортеж удобно использовать, когда набор значений должен рассматриваться как единое целое и не должен случайно изменяться в ходе работы программы.

Кортеж из одного элемента обязательно содержит запятую:

```
value = (10,)
```

Без запятой запись (10) будет обычным числом в круглых скобках, а не кортежем.

10. Распаковка кортежей и возврат нескольких значений

Элементы кортежа можно одновременно присвоить нескольким переменным. Такая операция называется распаковкой:

```
point = (10, 20)
x, y = point


print(x)
print(y)
```

Функция Python может возвращать несколько значений. На практике они возвращаются как кортеж:

```
def min_max(numbers):
    return min(numbers), max(numbers)


minimum, maximum = min_max([5, 2, 9, 1])
print(minimum, maximum)
```

Это позволяет удобно получать из одной функции несколько связанных результатов.

Список можно преобразовать в кортеж и обратно:

```
numbers = [1, 2, 3]
values = tuple(numbers)
new_numbers = list(values)
```

11. Встроенные функции для работы с последовательностями

Python содержит ряд встроенных функций, которые часто используются при обработке списков и кортежей.

```
numbers = [5, 8, 2, 10, 4]


print(len(numbers))
print(min(numbers))
print(max(numbers))
print(sum(numbers))
```

Функция sorted() возвращает новую отсортированную последовательность в виде списка:

```
sorted_numbers = sorted(numbers)
```

Для списков существует также метод sort(), который изменяет исходный список:

```
numbers.sort()
```

Это важное различие: sorted() создаёт новый результат, а list.sort() изменяет существующий список.

Для перебора элементов вместе с их индексами можно применять enumerate():

```
names = ["Анна", "Иван", "Мария"]


for index, name in enumerate(names):
    print(index, name)
```

12. Функции, принимающие последовательности

Список или кортеж можно передать в функцию как обычный аргумент. Это позволяет отделить алгоритм обработки данных от места, где эти данные были созданы.

```
def calculate_average(numbers):
    return sum(numbers) / len(numbers)
values = [10, 20, 30, 40]
result = calculate_average(values)
print(result)
```

Такой подход делает программу более структурированной и позволяет использовать одну функцию для обработки разных наборов данных.

13. Lambda-функции

Lambda-функция — небольшая анонимная функция, которая записывается в одну строку. Общий вид:

```
lambda параметры: выражение
```

Например:

```
square = lambda x: x ** 2
print(square(5))
```

Эта запись по смыслу близка к обычной функции:

```
def square(x):
    return x ** 2
```

Lambda-функции не следует рассматривать как замену обычным функциям. Они наиболее полезны, когда короткую функцию необходимо передать другой функции как аргумент.

Например, список можно отсортировать по абсолютному значению элементов:

```
numbers = [-10, 3, -2, 8, -5]
result = sorted(numbers, key=lambda x: abs(x))
print(result)
```

14. Функции как объекты

В Python функция является объектом. Её можно присвоить переменной и затем вызвать через новое имя:

```
def greet(name):
    print("Привет,", name)

function = greet
function("Иван")
```

Эта особенность позволяет передавать функции другим функциям. Именно поэтому встроенная функция sorted() может принимать функцию через параметр key, в том числе lambda-функцию.

ПРАКТИЧЕСКАЯ ЧАСТЬ

Задание 1: Создание и вызов функций

1. Создайте функцию greet(name), которая принимает имя пользователя и выводит приветствие.

2. Создайте функцию square(number), которая возвращает квадрат переданного числа.

3. Создайте функцию max_of_two(a, b), которая возвращает большее из двух значений без использования встроенной функции max().

4. Вызовите каждую функцию минимум два раза с различными аргументами и зафиксируйте результаты.

Задание 2: Аргументы и возвращаемые значения

1. Создайте функцию calculate_rectangle(width, height), которая возвращает площадь прямоугольника.

2. Добавьте функцию calculate_perimeter(width, height), возвращающую периметр.

3. Создайте функцию describe_rectangle(width, height, unit="см"), которая выводит размеры прямоугольника и использует значение единицы измерения по умолчанию.

4. Продемонстрируйте вызов функции describe_rectangle() как с аргументом unit, так и без него.

Задание 3: Области видимости переменных

Выполните программу:

```
value = 10
```

```
def example():
    value = 20
    print(value)
```

```
example()
print(value)
```

1. Зафиксируйте результат выполнения программы.

2. Объясните, почему выводятся два разных значения.

3. Добавьте внутри функции ещё одну локальную переменную и попробуйте обратиться к ней после вызова функции. Зафиксируйте и объясните полученный результат.

Задание 4: Работа со списками

1. Создайте список, содержащий не менее 10 целых чисел.

2. Выведите первый, третий и последний элементы.

3. Измените значение второго элемента.

4. Добавьте новый элемент с помощью append().

5. Удалите один из элементов с помощью remove() или pop().

6. Выведите итоговый список и количество его элементов.

Задание 5: Срезы

Используйте список:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

С помощью срезов получите и выведите:

• первые три элемента;

• последние три элемента;

• элементы с 3-го по 7-й;

• каждый второй элемент;

• список в обратном порядке.

Задание 6: Работа с кортежами

1. Создайте кортеж measurements, содержащий не менее 8 числовых значений.

2. Выведите первый, последний и несколько элементов с помощью среза.

3. Попробуйте изменить один элемент кортежа и зафиксируйте полученную ошибку.

4. Объясните, почему изменение невозможно.

5. Распакуйте первые два значения кортежа в отдельные переменные.

6. Преобразуйте кортеж в список, измените один элемент и преобразуйте результат обратно в кортеж.

Задание 7: Встроенные функции

Для созданного ранее списка чисел определите:

• количество элементов;

• сумму элементов;

• минимальное значение;

• максимальное значение;

• среднее арифметическое.

Получите отсортированную копию списка с помощью sorted(). После этого отсортируйте сам список методом sort(). Объясните различие между результатами этих операций.

Задание 8: Реализация собственных алгоритмов

Не используя встроенные функции sum(), min() и max(), реализуйте функции:

`calculate_sum(numbers)`
`find_min(numbers)`
`find_max(numbers)`

Каждая функция должна принимать список чисел и возвращать соответствующий результат. Сравните полученные значения с результатами встроенных функций Python.

Задание 9: Линейный поиск

Реализуйте функцию:

`find_element(numbers, target)`

Функция должна последовательно просматривать список и возвращать индекс первого найденного элемента. Если значение отсутствует, функция должна вернуть -1.

Проверьте работу алгоритма для существующего и отсутствующего значения.

Задание 10: Возврат нескольких значений

Создайте функцию analyze(numbers), которая принимает список чисел и возвращает кортеж из трёх значений:

• минимальное значение;

• максимальное значение;

• среднее арифметическое.

Вызовите функцию и распакуйте возвращённый кортеж в три отдельные переменные. Выведите результаты на экран.

Задание 11: Lambda-функции

Используйте список:

```
numbers = [-10, 3, -2, 8, -5]
```

1. Получите обычную сортировку списка с помощью sorted().

2. Получите сортировку по абсолютному значению элементов, используя параметр key и lambda-функцию.

3. Сравните два результата и объясните назначение lambda-функции в данном примере.

Задание 12: Самостоятельная работа

Разработайте программу для анализа набора результатов измерений. Исходные данные должны храниться в списке, содержащем не менее 10 числовых значений.

Программа должна содержать отдельные функции:

`calculate_average(measurements)`
`find_minimum(measurements)`
`find_maximum(measurements)`
`count_above_average(measurements)`
`find_value(measurements, target)`
`analyze_measurements(measurements)`

Функция analyze_measurements() должна возвращать кортеж, содержащий минимум, максимум и среднее значение. Полученный кортеж необходимо распаковать в отдельные переменные.

Программа должна вывести:

- количество измерений;
- минимальное значение;
- максимальное значение;
- среднее значение;
- количество значений выше среднего;
- результат поиска заданного пользователем значения.

В функциях calculate_average(), find_minimum() и find_maximum() запрещается использовать встроенные функции sum(), min() и max(). Необходимые алгоритмы требуется реализовать самостоятельно.

Требования к результату работы

После выполнения лабораторной работы студент должен уметь:

- создавать и вызывать функции;
- различать параметры и аргументы;
- использовать оператор return и объяснять его отличие от print();
- использовать позиционные, именованные аргументы и значения по умолчанию;
- объяснять различие локальных и глобальных областей видимости;
- создавать и изменять списки;
- создавать кортежи и объяснять их отличие от списков;
- использовать индексацию, отрицательные индексы и срезы;
- распаковывать кортежи и возвращать из функции несколько значений;
- использовать len(), min(), max(), sum(), sorted(), abs(), round() и enumerate();
- реализовывать простейшие алгоритмы суммирования, поиска минимума и максимума, подсчёта и линейного поиска;
- использовать lambda-функции в простых задачах.

Контрольные вопросы для самопроверки

1. Что такое функция и для чего она используется?
2. Чем определение функции отличается от её вызова?
3. В чём разница между параметром и аргументом функции?

4. Для чего используется оператор return?

5. Чем return отличается от print()?

6. Что происходит с выполнением функции после выполнения return?

7. Что такое аргумент по умолчанию?

8. Что такое именованный аргумент?

9. Что называется областью видимости переменной?

10. Чем локальная переменная отличается от глобальной?

11. Почему использование большого количества глобальных переменных нежелательно?

12. Что такое список в Python?

13. С какого значения начинается индексация списка?

14. Для чего используются отрицательные индексы?

15. Какими способами можно добавить и удалить элементы списка?

16. Что такое срез и что означают start, stop и step?

17. Почему элемент с индексом stop не входит в срез?

18. Что такое кортеж?

19. В чём основное отличие кортежа от списка?

20. Как создать кортеж, содержащий один элемент?

21. Что такое распаковка кортежа?

22. Как функция может вернуть несколько значений?

23. Как преобразовать список в кортеж и кортеж в список?

24. В чём различие между sorted() и методом list.sort()?

25. Для чего используется enumerate()?

26. Как работает алгоритм последовательного суммирования элементов?

27. Как можно найти максимальное значение без использования max()?

28. Что такое линейный поиск?

29. Что должна вернуть функция поиска, если значение отсутствует?

30. Что такое lambda-функция?

31. В каких ситуациях использование lambda-функции удобно?

32. Что означает утверждение «функция в Python является объектом»?

Лабораторная работа №4: Организация Python-проекта

Цель работы: изучить принципы организации Python-проекта, научиться разделять программу на модули и пакеты, использовать различные способы импорта, работать с файловой системой и операционной системой, обрабатывать исключения, применять логирование, использовать конфигурационные файлы, создавать виртуальные окружения и управлять зависимостями проекта.

В ходе выполнения лабораторной работы необходимо научиться организовывать структуру проекта, распределять программный код между несколькими файлами, корректно обрабатывать ошибки, сохранять диагностическую информацию в журнал, выносить изменяемые параметры в конфигурационный файл и создавать воспроизводимое окружение проекта.

ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1. Организация Python-проекта

На начальном этапе изучения Python программы часто размещаются в одном файле main.py. Для небольших учебных программ такой подход допустим. Однако по мере роста программы количество функций, настроек и вспомогательного кода увеличивается.

Если весь код находится в одном файле, программу становится сложнее читать, изменять, тестировать и повторно использовать. Поэтому исходный код принято разделять на отдельные логические части.

Пример структуры проекта:

```
measurement_project/
    main.py
    config.json
    requirements.txt
    .gitignore
    utils/
        __init__.py
        reader.py
        statistics.py
    data/
```

```
        measurements.txt
    logs/
```

В данном проекте main.py содержит точку входа в программу, пакет utils — функции чтения и обработки данных, config.json — изменяемые настройки, data — входные данные, а logs — журналы работы программы.

2. Модули Python

Модулем в Python может выступать обычный файл с расширением .py. Модули позволяют распределять функции и другие объекты программы между несколькими файлами.

Например, файл statistics.py может содержать функцию:

```
def calculate_average(numbers):
    return sum(numbers) / len(numbers)
```

Чтобы использовать функцию в другом файле, модуль необходимо импортировать:

```
import statistics
```

После этого обращение к объекту выполняется через имя модуля:

```
result = statistics.calculate_average([10, 20, 30])
```

3. Способы импорта

Python поддерживает несколько основных способов импорта.

Импорт всего модуля:

```
import math
print(math.sqrt(25))
```

Импорт отдельного объекта:

```
from math import sqrt
print(sqrt(25))
```

Назначение сокращённого имени модулю:

```
import math as m
print(m.pi)
```

Также можно переименовать отдельный импортируемый объект:

```
from math import sqrt as square_root
```

Конструкция from module import * импортирует множество имён в текущее пространство имён. Такой вариант обычно не рекомендуется,

поскольку затрудняет понимание происхождения объектов и может приводить к конфликтам имён.

4. Точка входа в программу и переменная __name__

Один и тот же Python-файл может быть запущен непосредственно или импортирован как модуль. Для определения способа использования файла существует специальная переменная __name__.

Если файл запущен непосредственно, значение __name__ равно строке "__main__". Поэтому точку входа часто оформляют следующим образом:

```
def main():
    print("Программа запущена")


if __name__ == "__main__":
    main()
```

Если такой файл импортируется из другого модуля, функция main() автоматически не вызывается. Это позволяет использовать определённые в файле функции без запуска всей программы.

5. Пакеты Python

Если проект содержит несколько связанных модулей, их можно объединить в пакет — каталог с Python-модулями.

```
utils/
    __init__.py
    reader.py
    statistics.py
```

Функцию из модуля пакета можно импортировать следующим образом:

```
from utils.statistics import calculate_average
```

В данной лабораторной работе используется обычный пакет с файлом __init__.py. Этот файл может быть пустым или содержать код инициализации пакета. В современных версиях Python существуют также namespace packages, которые могут работать без __init__.py. В рамках данной лабораторной работы мы используем классический вариант пакета с файлом __init__.py.

6. Работа с путями и файловой системой

Для работы с путями в современных версиях Python удобно использовать модуль pathlib и класс Path.

```
from pathlib import Path
```

Получение текущего рабочего каталога:

```
current_directory = Path.cwd()
print(current_directory)
```

Формирование пути к файлу:

```
file_path = Path("data") / "measurements.txt"
```

Проверка существования файла:

```
print(file_path.exists())
```

Создание каталога, если он отсутствует:

```
logs_dir = Path("logs")
logs_dir.mkdir(exist_ok=True)
```

Использование Path позволяет формировать пути без ручного объединения строк и делает код более переносимым между операционными системами.

7. Модуль os

Стандартный модуль os предоставляет дополнительные средства взаимодействия с операционной системой.

```
import os
print(os.getcwd())
```

С помощью os.environ можно получить значения переменных окружения:

```
print(os.environ.get("USERNAME"))
```

Для работы именно с путями в рамках данной лабораторной работы рекомендуется использовать pathlib, а os рассматривается как средство взаимодействия с окружением операционной системы.

8. Исключения

Во время выполнения программы могут возникать ошибки. Например, попытка преобразовать строку abc в число приведёт к исключению ValueError.

Для обработки ожидаемых ошибок используется конструкция try-except:

```
try:
    number = int(input("Введите число: "))
except ValueError:
    print("Некорректное число")
```

Если ошибка не возникает, блок except не выполняется. Если возникает указанное исключение, программа передаёт управление соответствующему обработчику.

9. Обработка нескольких типов исключений

В одной конструкции можно обрабатывать несколько различных ошибок:

```
try:
    with open("data.txt", "r", encoding="utf-8") as file:
        number = float(file.readline())
except FileNotFoundError:
    print("Файл не найден")
except ValueError:
    print("Файл содержит некорректные данные")
```

Обработка конкретных классов исключений предпочтительнее конструкции except без указания типа ошибки.

10. Блоки else и finally

Конструкция обработки исключений может содержать блоки else и finally:

```
try:
    number = int(input("Введите число: "))
except ValueError:
    print("Ошибка ввода")
else:
    print("Число успешно получено")
finally:
    print("Операция завершена")
```

Блок else выполняется только при отсутствии исключения. Блок finally выполняется независимо от результата выполнения блока try.

11. Создание исключений с помощью raise

Иногда программа должна самостоятельно определить, что полученные данные недопустимы. Для явного создания исключения используется оператор raise.

```
def validate_numbers(numbers):
    if len(numbers) == 0:
        raise ValueError("Список измерений пуст")
```

Оператор raise позволяет сообщить вызывающему коду о некорректном состоянии данных и передать обработку ошибки на более подходящий уровень программы.

12. Логирование

Для вывода промежуточной информации во время обучения часто используется print(). В проекте диагностическую информацию удобнее регистрировать с помощью стандартного модуля logging.

```
import logging

logging.basicConfig(level=logging.INFO)
logging.info("Программа запущена")
logging.warning("Предупреждение")
logging.error("Произошла ошибка")
```

Основные уровни логирования:

- DEBUG — подробная отладочная информация;
- INFO — обычные события работы программы;
- WARNING — потенциально проблемные ситуации;
- ERROR — ошибки;
- CRITICAL — критические ошибки.

13. Запись логов в файл

Логи можно сохранять в файл, чтобы информация о работе программы оставалась доступна после её завершения.

```
logging.basicConfig(
    filename="logs/app.log",
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s"
)
```

В журнале можно фиксировать запуск программы, чтение данных, количество обработанных значений, предупреждения и ошибки.

14. Конфигурационные файлы

Параметры, которые могут изменяться независимо от исходного кода, удобно хранить в отдельном конфигурационном файле. В данной лабораторной работе используется формат JSON.

Пример config.json:

```
{
    "input_file": "data/measurements.txt",
    "log_file": "logs/app.log"
}
```

Для чтения JSON используется стандартный модуль json:

```
import json

with open("config.json", "r", encoding="utf-8") as file:
    config = json.load(file)

input_file = config["input_file"]
```

Вынос настроек в отдельный файл позволяет менять пути и другие параметры без изменения исходного кода программы.

15. Виртуальные окружения

Разные Python-проекты могут использовать разные наборы и версии сторонних библиотек. Для изоляции зависимостей используются виртуальные окружения.

Создание окружения с помощью стандартного модуля venv:

```
python -m venv .venv
```

После выполнения команды в каталоге проекта создаётся директория .venv с отдельным окружением Python.

16. Активация виртуального окружения

В Windows PowerShell окружение активируется командой:

```
.venv\Scripts\Activate.ps1
```

В Windows CMD:

```
.venv\Scripts\activate
```

В Linux и macOS:

```
source .venv/bin/activate
```

После активации имя окружения обычно отображается в начале строки терминала. Для выхода из окружения используется команда deactivate.

17. Управление зависимостями с помощью pip

Для установки и управления сторонними библиотеками используется менеджер пакетов pip. Рекомендуется запускать его через текущий интерпретатор Python:

```
python -m pip
```

Просмотр пакетов, установленных в текущем окружении:

```
python -m pip list
```

Установка конкретной зависимости выполняется командой python -m pip install <имя_пакета>. В рамках данной лабораторной работы специально устанавливать сторонний пакет не требуется: основная задача — понять изоляцию окружения и механизм фиксации зависимостей.

18. Файл requirements.txt

Список зависимостей окружения можно сохранить в файл requirements.txt:

```
python -m pip freeze > requirements.txt
```

Зависимости из такого файла устанавливаются командой:

```
python -m pip install -r requirements.txt
```

Если проект пока не использует сторонние библиотеки, requirements.txt может оказаться пустым. Это является нормальным результатом и показывает, что проект основан только на стандартной библиотеке Python.

19. Файл .gitignore

Некоторые файлы и каталоги проекта не следует добавлять в Git-репозиторий. Для исключения таких файлов используется .gitignore.

```
.venv/
__pycache__/
*.pyc
*.log
```

Виртуальное окружение обычно не хранится в Git. Вместо него передаются исходный код проекта и файл зависимостей, по которому окружение можно воспроизвести.

ПРАКТИЧЕСКАЯ ЧАСТЬ

В практической части все задания выполняются последовательно в рамках одного проекта measurement_project. Не создавайте отдельный проект для каждого задания. Каждый следующий этап должен дополнять результат предыдущего.

Задание 1: Создание проекта

1. Создайте каталог measurement_project и откройте его в Visual Studio Code.
2. Создайте файл main.py.
3. Создайте каталоги utils, data и logs.
4. В каталоге utils создайте файлы __init__.py, reader.py и statistics.py.
5. Создайте в корне проекта файлы config.json и .gitignore.
6. Зафиксируйте итоговую структуру проекта в отчёте.

На данном этапе структура должна иметь вид:

```
measurement_project/
    main.py
    config.json
    .gitignore
```

```
    utils/
        __init__.py
        reader.py
        statistics.py
    data/
    logs/
```

Задание 2: Модуль обработки данных

1. В файле utils/statistics.py создайте функцию calculate_average(numbers), возвращающую среднее арифметическое.

2. Добавьте функции find_min(numbers) и find_max(numbers).

3. Импортируйте функции в main.py с помощью конструкции from utils.statistics import ...

4. Создайте тестовый список чисел и продемонстрируйте работу функций.

Задание 3: Точка входа

1. В main.py создайте функцию main().

2. Перенесите запуск тестовой обработки данных внутрь main().

3. Добавьте конструкцию if __name__ == "__main__": main().

4. Создайте временный файл test_import.py и импортируйте main.py как модуль.

5. Убедитесь, что функция main() не запускается автоматически при импорте, и объясните результат.

Задание 4: Чтение измерений из файла

1. Создайте файл data/measurements.txt и запишите в него не менее 10 числовых значений, каждое с новой строки.

2. В utils/reader.py создайте функцию read_numbers(file_path).

3. Функция должна открыть файл, прочитать строки, преобразовать каждую строку в float и вернуть список чисел.

4. Импортируйте read_numbers() в main.py и используйте полученные данные вместо тестового списка.

Задание 5: Работа с pathlib

1. Подключите from pathlib import Path.
2. Используйте Path для формирования пути к входному файлу.
3. Выведите текущий рабочий каталог.
4. Проверьте существование файла measurements.txt с помощью exists().
5. Обеспечьте создание каталога logs с помощью mkdir(exist_ok=True).

Задание 6: Обработка исключений

1. Добавьте обработку FileNotFoundError при чтении файла.
2. Добавьте обработку ValueError, возникающего при наличии строки, которую невозможно преобразовать в float.
3. Проверьте работу программы с корректным файлом.
4. Измените путь так, чтобы файл отсутствовал, и зафиксируйте результат обработки ошибки.
5. Добавьте в файл некорректную строку и зафиксируйте результат обработки ValueError.

Задание 7: Проверка данных с помощью raise

1. Создайте функцию validate_numbers(numbers).
2. Если список пуст, выполните raise ValueError("Список измерений пуст").
3. Вызовите функцию после чтения данных.
4. Обработайте возникшее исключение в основной логике программы.

Задание 8: Логирование

1. Подключите модуль logging.
2. Настройте запись сообщений в logs/app.log.
3. Используйте формат, содержащий дату и время, уровень сообщения и текст.
4. Записывайте в журнал запуск программы, успешное чтение файла, количество прочитанных значений, возникшие ошибки и завершение программы.
5. Продемонстрируйте содержимое app.log в отчёте.

Задание 9: Конфигурационный файл

Создайте config.json со следующим содержимым:

```
{
    "input_file": "data/measurements.txt",
    "log_file": "logs/app.log"
}
```

1. Подключите стандартный модуль json.

2. Загрузите настройки из config.json.

3. Измените программу так, чтобы путь к файлу измерений получался из конфигурации, а не задавался непосредственно в main.py.

4. Проверьте, что после изменения input_file в config.json программа использует новое значение без изменения исходного кода.

Задание 10: Виртуальное окружение

1. В корне проекта создайте окружение командой python -m venv .venv.

2. Активируйте созданное окружение.

3. Выполните python --version.

4. Определите путь к используемому интерпретатору с помощью where python в Windows или which python в Linux/macOS.

5. Зафиксируйте в отчёте, что используется интерпретатор из каталога .venv.

Задание 11: Фиксация зависимостей

1. В активированном окружении выполните python -m pip list.

2. Создайте requirements.txt командой python -m pip freeze > requirements.txt.

3. Откройте полученный файл.

4. Если файл пуст, объясните, почему это является корректным результатом для проекта, использующего только стандартную библиотеку Python.

Задание 12: Настройка .gitignore

Добавьте в .gitignore:

```
.venv/
__pycache__/
*.pyc
*.log
```

1. Инициализируйте Git-репозиторий, если текущий проект ещё не является репозиторием.

2. Выполните git status.

3. Убедитесь, что содержимое .venv и файлы *.log не предлагаются для добавления в репозиторий.

4. Зафиксируйте результат в отчёте.

Задание 13: Итоговая самостоятельная работа

Доведите проект measurement_project до законченного состояния. В результате он должен иметь следующую структуру:

```
measurement_project/
    main.py
    config.json
    requirements.txt
    .gitignore
    utils/
        __init__.py
        reader.py
        statistics.py
    data/
        measurements.txt
    logs/
```

Программа должна:

1. получать путь к файлу данных из config.json;

2. читать числовые значения из файла через функцию модуля utils.reader;

3. проверять наличие и корректность данных;

4. обрабатывать FileNotFoundError и ValueError;

5. вычислять количество значений, минимум, максимум и среднее значение с использованием функций из utils.statistics;

6. записывать основные действия и ошибки в лог;

7. использовать функцию main() и конструкцию if __name__ == "__main__";

8. корректно запускаться внутри созданного виртуального окружения;

9. иметь requirements.txt и .gitignore;

10. быть проверенной минимум в трёх сценариях: корректный файл, отсутствующий файл и файл с некорректным значением.

Последовательность окончательной реализации и организация вызовов функций определяются студентом самостоятельно.

Требования к результату работы

После выполнения лабораторной работы студент должен уметь:

• объяснять назначение модулей и пакетов;

• использовать основные способы импорта;

• организовывать точку входа с помощью if __name__ == "__main__";

• структурировать Python-проект из нескольких файлов и каталогов;

• работать с путями с помощью pathlib;

• обрабатывать исключения с помощью try, except, else и finally;

• создавать исключения с помощью raise;

• использовать logging и записывать журналы в файл;

• загружать настройки из JSON-конфигурации;

• создавать и активировать виртуальное окружение;

• использовать pip и requirements.txt;

• объяснять назначение .gitignore и состав файлов, которые не следует хранить в репозитории.

Контрольные вопросы для самопроверки

1. Что такое модуль Python и для чего он используется?

2. Чем модуль отличается от пакета?

3. Какие основные способы импорта существуют в Python?

4. Почему from module import * обычно не рекомендуется?

5. Для чего используется конструкция if __name__ == "__main__"?

6. Чем запуск файла отличается от его импорта как модуля?

7. Для чего используется pathlib?

8. Что такое исключение?

9. Как работает конструкция try-except?

10. Для чего используются else и finally при обработке исключений?

11. Для чего используется оператор raise?

12. Чем логирование отличается от вывода информации через print()?

13. Какие основные уровни логирования существуют?

14. Для чего используются конфигурационные файлы?

15. Почему изменяемые параметры программы желательно выносить из исходного кода?

16. Что такое виртуальное окружение Python и зачем оно используется?

17. Как создать и активировать виртуальное окружение?

18. Для чего используется pip?

19. Для чего нужен файл requirements.txt?

20. Как восстановить зависимости проекта из requirements.txt?

21. Для чего используется .gitignore?

22. Почему .venv, __pycache__ и файлы логов обычно не добавляются в Git?

Лабораторная работа №5: Объектно-ориентированное программирование в Python

Цель работы: изучить основные принципы объектно-ориентированного программирования в Python; научиться создавать классы и объекты, определять атрибуты и методы, использовать конструктор класса, применять инкапсуляцию, наследование, абстракцию и полиморфизм, а также переопределять методы родительских классов.

В ходе выполнения лабораторной работы необходимо разработать небольшой объектно-ориентированный проект управления транспортными средствами, последовательно применяя изучаемые возможности языка Python.

ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1. Объектно-ориентированное программирование

Объектно-ориентированное программирование (ООП) — подход к разработке программ, при котором данные и операции над ними объединяются в объекты. Каждый объект имеет состояние, определяемое его атрибутами, и поведение, определяемое методами.

Например, транспортное средство можно представить объектом, хранящим марку, модель и скорость и предоставляющим операции разгона и торможения.

```
class Vehicle:
    def __init__(self, brand, speed=0):
        self.brand = brand
        self.speed = speed

    def accelerate(self, value):
        self.speed += value
```

К основным принципам ООП относят инкапсуляцию, наследование, полиморфизм и абстракцию.

2. Классы и объекты

Класс — описание структуры и поведения объектов определённого типа. Объект — конкретный экземпляр класса.

```
class Vehicle:
    pass

car1 = Vehicle()
car2 = Vehicle()

print(car1 is car2)
```

Оба объекта созданы на основе одного класса, но являются разными экземплярами и могут иметь собственное состояние.

3. Конструктор __init__ и параметр self

Метод __init__ вызывается при создании объекта и обычно используется для начальной настройки его атрибутов.

```
class Vehicle:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year
```

Параметр self содержит ссылку на конкретный объект, для которого выполняется метод. Через self методы получают доступ к атрибутам и другим методам этого объекта.

```
car = Vehicle("Toyota", "Camry", 2024)
print(car.brand)
print(car.model)
```

4. Атрибуты и методы

Атрибуты хранят состояние объекта. Методы определяют действия, которые объект может выполнять.

```
class Vehicle:
    def __init__(self, brand, speed=0):
        self.brand = brand
        self.speed = speed

    def accelerate(self, value):
        self.speed += value

    def stop(self):
        self.speed = 0
```

Метод вызывается через объект:

```
car.accelerate(20)
print(car.speed)
car.stop()
```

5. Атрибуты экземпляра и атрибуты класса

Атрибуты экземпляра создаются для конкретного объекта, обычно через self. Атрибут класса принадлежит самому классу и изначально является общим для всех его экземпляров.

```
class Vehicle:
    category = "Транспорт"

    def __init__(self, brand):
        self.brand = brand
```

В данном примере category является атрибутом класса, а brand — атрибутом экземпляра.

6. Метод __str__

Специальный метод __str__ определяет удобное текстовое представление объекта. Он автоматически используется, например, при передаче объекта функции print().

```
def __str__(self):
    return f"{self.brand} {self.model}, скорость: {self.speed}
км/ч"
print(car)
```

7. Инкапсуляция

Инкапсуляция предполагает объединение данных и методов работы с ними внутри объекта и управление доступом к внутреннему состоянию через публичный интерфейс класса.

В Python нет строгого модификатора private, аналогичного некоторым другим языкам. Вместо этого используются соглашения об именовании и механизмы свойств.

Имя с одним символом подчёркивания, например _speed, означает по соглашению, что атрибут предназначен преимущественно для внутреннего использования. Python при этом технически не запрещает прямой доступ к нему.

Имя, начинающееся с двух символов подчёркивания, например __speed, подвергается преобразованию имени (name mangling). Это снижает вероятность случайного обращения к атрибуту и конфликтов имён в классах-наследниках, но не делает атрибут абсолютно недоступным.

8. Свойства property

Свойства позволяют сохранить простой синтаксис обращения к атрибуту, но при этом контролировать чтение и изменение значения.

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand
        self._speed = 0

    @property
    def speed(self):
        return self._speed

    @speed.setter
    def speed(self, value):
        if value < 0:
            raise ValueError("Скорость не может быть
отрицательной")
        self._speed = value
```

Внешний код продолжает обращаться к свойству как к обычному атрибуту:

```
car.speed = 80
print(car.speed)
```

9. Наследование

Наследование позволяет создать новый класс на основе уже существующего. Класс-наследник получает доступ к поведению базового класса и может расширять или изменять его.

```
class Vehicle:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

class Car(Vehicle):
    pass
```

Объект Car получает атрибуты и методы Vehicle, если они не переопределены в классе-наследнике.

10. Использование super()

Функция super() позволяет обратиться к реализации базового класса. Она часто используется при расширении конструктора.

```
class Car(Vehicle):
    def __init__(self, brand, model, doors):
        super().__init__(brand, model)
        self.doors = doors
```

Вызов super().__init__() позволяет повторно использовать инициализацию базовой части объекта и не дублировать код.

11. Переопределение методов

Переопределение метода означает создание в классе-наследнике метода с тем же именем, что и в базовом классе, но с другим поведением.

```
class Vehicle:
    def get_info(self):
        return f"{self.brand} {self.model}"

class Car(Vehicle):
    def get_info(self):
        return f"Автомобиль: {self.brand} {self.model}, дверей:
{self.doors}"
```

При необходимости переопределённый метод может использовать результат родительской реализации через super().

12. Полиморфизм

Полиморфизм позволяет использовать одинаковый интерфейс для объектов разных классов. Один и тот же вызов метода может приводить к разному поведению в зависимости от фактического типа объекта.

```
class Car(Vehicle):
    def move(self):
        return "Автомобиль движется по дороге"

class Motorcycle(Vehicle):
    def move(self):
        return "Мотоцикл движется по дороге"
vehicles = [car, motorcycle]

for vehicle in vehicles:
    print(vehicle.move())
```

Код цикла не проверяет тип каждого объекта. Он использует общий метод move(), а конкретная реализация выбирается самим объектом.

13. Абстракция

Абстракция позволяет выделить существенные свойства объектов и определить общий интерфейс, не задавая все детали реализации в базовом классе.

Для создания абстрактных классов стандартная библиотека Python предоставляет модуль abc.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def move(self):
        pass
```

Класс, содержащий нереализованные абстрактные методы, нельзя использовать для создания обычного экземпляра. Класс-наследник должен реализовать требуемые методы.

```
class Car(Vehicle):
    def move(self):
        return "Автомобиль движется"
```

Абстрактный класс таким образом задаёт общий контракт для группы классов.

14. Взаимодействие объектов и композиция

Объекты в программе могут взаимодействовать друг с другом. Один объект может хранить ссылки на другие объекты и использовать их методы. Такой подход часто называют композицией.

```
class Garage:
    def __init__(self):
        self.vehicles = []

    def add_vehicle(self, vehicle):
        self.vehicles.append(vehicle)

    def show_all(self):
        for vehicle in self.vehicles:
            print(vehicle)
```

Garage не является транспортным средством и поэтому не наследуется от Vehicle. Вместо этого он содержит коллекцию объектов транспортных средств.

ПРАКТИЧЕСКАЯ ЧАСТЬ

В практической части все задания выполняются последовательно в рамках одного проекта vehicle_project. Каждый следующий этап должен дополнять результат предыдущего.

Задание 1: Создание структуры проекта

1. Создайте каталог vehicle_project и откройте его в Visual Studio Code.
2. Создайте файл main.py и каталог models.
3. В каталоге models создайте файлы __init__.py и vehicle.py.
4. Создайте в корне проекта файл garage.py.
5. Зафиксируйте полученную структуру проекта в отчёте.

```
vehicle_project/
    main.py
    garage.py
    models/
        __init__.py
        vehicle.py
```

Задание 2: Базовый класс Vehicle

1. В файле models/vehicle.py создайте класс Vehicle.
2. Реализуйте конструктор, принимающий brand, model и year.
3. Добавьте атрибут _speed со значением 0.
4. Создайте два объекта Vehicle в main.py и выведите значения их атрибутов.
5. Убедитесь, что объекты имеют независимое состояние.

Задание 3: Методы объекта

1. Добавьте методы accelerate(value), brake(value), stop() и get_info().
2. Метод accelerate() должен увеличивать скорость на переданное значение.
3. Метод brake() должен уменьшать скорость, но итоговое значение не должно становиться меньше нуля.
4. Метод stop() должен устанавливать скорость равной нулю.
5. Продемонстрируйте работу методов в main.py.

Задание 4: Инкапсуляция с помощью property

1. Добавьте свойство speed, возвращающее значение _speed.
2. Добавьте setter для speed.
3. При попытке установить отрицательную скорость выбрасывайте ValueError.
4. Измените методы accelerate() и brake() так, чтобы они использовали свойство speed.
5. Проверьте корректное изменение скорости и обработку попытки присвоить отрицательное значение.

Задание 5: Метод __str__

1. Реализуйте __str__() для класса Vehicle.
2. Строковое представление должно содержать марку, модель, год и текущую скорость.
3. Выведите объект функцией print() и убедитесь, что используется созданное представление.

Задание 6: Наследование и класс Car

1. Создайте файл models/car.py.
2. Реализуйте класс Car, наследующийся от Vehicle.
3. Добавьте собственный атрибут doors.
4. Используйте super().__init__() для инициализации общей части объекта.
5. Переопределите get_info(), добавив количество дверей.
6. Создайте объект Car и продемонстрируйте работу унаследованных и переопределённых методов.

Задание 7: Классы Truck и Motorcycle

1. Создайте файлы models/truck.py и models/motorcycle.py.
2. Создайте класс Truck с дополнительным атрибутом max_load.
3. Создайте класс Motorcycle с дополнительным атрибутом has_sidecar.
4. Оба класса должны наследоваться от Vehicle и использовать super().
5. Переопределите get_info() с учётом собственных атрибутов классов.

Задание 8: Абстрактный метод move()

1. Измените Vehicle так, чтобы класс наследовался от ABC.
2. Добавьте абстрактный метод move() с декоратором @abstractmethod.
3. Реализуйте move() в Car, Truck и Motorcycle.
4. Попробуйте создать объект Vehicle напрямую и зафиксируйте полученный результат.
5. Объясните, почему после добавления абстрактного метода экземпляр Vehicle создать нельзя.

Задание 9: Полиморфизм

1. Создайте список vehicles, содержащий объекты Car, Truck и Motorcycle.
2. В одном цикле вызовите move() для каждого объекта.
3. В этом же цикле выведите get_info() для каждого объекта.
4. Не используйте проверки isinstance() или сравнение типов для выбора реализации метода.
5. Объясните, каким образом Python выбирает нужную реализацию move() и get_info().

Задание 10: Класс Garage

1. В файле garage.py создайте класс Garage.
2. В конструкторе создайте пустой список vehicles.
3. Реализуйте add_vehicle(vehicle), remove_vehicle(vehicle), show_all() и find_by_brand(brand).
4. Добавьте в гараж объекты разных классов.
5. Выведите все транспортные средства и выполните поиск по марке.
6. Класс Garage не должен содержать отдельную логику для Car, Truck и Motorcycle.

Задание 11: Итоговая структура проекта

После выполнения основных этапов проект должен иметь следующую структуру:

```
vehicle_project/
    main.py
    garage.py
    models/
        __init__.py
        vehicle.py
        car.py
        truck.py
        motorcycle.py
```

1. Организуйте все импорты между модулями.
2. Разместите демонстрационный сценарий внутри функции main().

3. Добавьте конструкцию if __name__ == "__main__": main().

4. Убедитесь, что проект запускается из корневого каталога без ошибок.

Задание 12: Самостоятельная работа

Самостоятельно добавьте в проект ещё один класс транспортного средства, например Bus, Boat, Bicycle или ElectricCar. Тип транспортного средства выбирается студентом самостоятельно.

Новый класс должен:

1. наследоваться от Vehicle;
2. содержать минимум два собственных атрибута;
3. использовать super() при инициализации;
4. переопределять get_info();
5. реализовывать абстрактный метод move();
6. корректно работать в общем списке vehicles;
7. добавляться в Garage и отображаться методом show_all() без изменения внутренней логики класса Garage.

Продемонстрируйте, что добавление нового класса не требует добавления специальных проверок его типа в код, работающий через общий интерфейс Vehicle.

Требования к результату работы

После выполнения лабораторной работы студент должен уметь:

- объяснять различие между классом и объектом;
- создавать классы и экземпляры классов;
- использовать __init__ и объяснять назначение self;
- создавать атрибуты и методы экземпляра;
- различать атрибуты экземпляра и класса;
- использовать __str__ для текстового представления объекта;
- применять инкапсуляцию и свойства property;
- создавать классы-наследники и использовать super();
- переопределять методы;
- объяснять и применять полиморфизм;

• создавать абстрактные классы с ABC и abstractmethod;

• организовывать взаимодействие объектов через композицию;

• распределять классы между модулями проекта.

Контрольные вопросы для самопроверки

1. Что такое класс и что такое объект?

2. Для чего используется метод __init__?

3. Что представляет собой параметр self?

4. Что такое атрибут объекта?

5. Чем метод экземпляра отличается от обычной функции?

6. Чем атрибут экземпляра отличается от атрибута класса?

7. Для чего используется метод __str__?

8. Что понимается под инкапсуляцией?

9. Что означает один символ подчёркивания перед именем атрибута в Python?

10. Что происходит с именами атрибутов, начинающихся с двух символов подчёркивания?

11. Для чего используется property?

12. Что такое наследование?

13. Для чего используется super()?

14. Что называется переопределением метода?

15. Что такое полиморфизм?

16. Как полиморфизм проявляется при работе с объектами разных классов через общий метод?

17. Что понимается под абстракцией?

18. Для чего используются ABC и @abstractmethod?

19. Можно ли создать экземпляр класса, содержащего нереализованный абстрактный метод?

20. Чем композиция отличается от наследования?

21. Почему класс Garage не должен проверять конкретный тип каждого транспортного средства?

22. Почему общий интерфейс базового класса упрощает добавление новых типов объектов?

Лабораторная работа №6: Разработка GUI-приложений на PyQt6

Цель работы: изучить основные принципы разработки графических пользовательских интерфейсов средствами PyQt6, научиться создавать окна приложения, размещать и настраивать виджеты, использовать менеджеры компоновки, обрабатывать действия пользователя с помощью механизма сигналов и слотов и разрабатывать простое настольное приложение.

В ходе выполнения лабораторной работы необходимо создать законченное графическое приложение «Менеджер задач». Проект должен развиваться последовательно на протяжении всей практической части: от создания пустого окна до приложения, позволяющего добавлять, удалять и учитывать задачи, обрабатывать некорректные действия пользователя и изменять состояние интерфейса.

ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1. Графический пользовательский интерфейс

Графический пользовательский интерфейс (Graphical User Interface, GUI) — способ взаимодействия пользователя с программой при помощи визуальных элементов: окон, кнопок, полей ввода, списков, переключателей, меню и других компонентов.

В консольной программе порядок выполнения обычно непосредственно задаётся последовательностью инструкций. В графическом приложении значительная часть действий выполняется в ответ на события, возникающие во время работы программы.

```
Пользователь нажал кнопку
        ↓
Возникло событие
        ↓
Вызван обработчик
        ↓
Изменилось состояние интерфейса
```

Такой подход называется событийно-ориентированным программированием. Приложение большую часть времени ожидает действия

пользователя или другие события и выполняет соответствующий код только после их возникновения.

2. Qt и PyQt6

Qt — набор библиотек и инструментов для разработки графических и кроссплатформенных приложений. PyQt6 предоставляет возможность использовать возможности Qt из языка Python.

В данной лабораторной работе используется библиотека PyQt6. Она является внешней зависимостью проекта и устанавливается в виртуальное окружение с помощью pip:

```
python -m pip install PyQt6
```

После установки PyQt6 библиотеку необходимо зафиксировать в файле requirements.txt, чтобы окружение проекта можно было воспроизвести на другом компьютере.

3. Структура минимального PyQt6-приложения

Любое простое PyQt6-приложение содержит объект QApplication, хотя бы одно окно и основной цикл обработки событий.

```
import sys


from PyQt6.QtWidgets import QApplication, QWidget


app = QApplication(sys.argv)
window = QWidget()
window.show()
sys.exit(app.exec())
```

QApplication представляет запущенное графическое приложение и управляет взаимодействием с операционной системой. Метод show() делает окно видимым, а app.exec() запускает цикл обработки событий. Пока этот цикл выполняется, программа ожидает действия пользователя и не завершается сразу после создания окна.

4. Виджеты

Виджет — элемент графического интерфейса. Само окно также является виджетом, а внутри него могут располагаться другие элементы. В рамках лабораторной работы используются несколько базовых компонентов PyQt6.

4.1. QLabel

QLabel предназначен для отображения текста или другой информации в окне.

```
label = QLabel("Введите задачу")
```

4.2. QLineEdit

QLineEdit представляет однострочное поле ввода. Текущее содержимое поля можно получить методом text(), а очистить поле — методом clear().

```
self.task_input = QLineEdit()
self.task_input.setPlaceholderText("Название задачи")

text = self.task_input.text()
self.task_input.clear()
```

4.3. QPushButton

QPushButton используется для создания кнопки. Основным событием кнопки является нажатие пользователем.

```
self.add_button = QPushButton("Добавить")
```

4.4. QListWidget

QListWidget позволяет отображать список элементов и выбирать один из них. В приложении «Менеджер задач» этот виджет используется для хранения списка задач.

```
self.task_list = QListWidget()
self.task_list.addItem("Изучить PyQt")
```

5. Свойства и методы виджетов

Состояние элементов интерфейса можно получать и изменять во время выполнения программы. Например, приложение может изменять текст подписи, включать и выключать кнопку или менять заголовок окна.

```
self.setWindowTitle("Менеджер задач")
```

```
self.resize(500, 400)
self.counter_label.setText("Количество задач: 3")
self.add_button.setEnabled(False)
```

Методы конкретного виджета зависят от его назначения. Поэтому при разработке GUI важно понимать не только внешний вид элемента, но и способы чтения и изменения его состояния.

6. Менеджеры компоновки

Расположение элементов интерфейса можно задавать вручную через координаты, однако такой подход плохо адаптируется к изменению размера окна, размера шрифта и параметров операционной системы. Для организации интерфейса в Qt используются менеджеры компоновки (layout).

В лабораторной работе используются QVBoxLayout и QHBoxLayout. QVBoxLayout размещает элементы вертикально, а QHBoxLayout — горизонтально.

```
main_layout = QVBoxLayout()
main_layout.addWidget(self.task_input)
main_layout.addWidget(self.task_list)

buttons_layout = QHBoxLayout()
buttons_layout.addWidget(self.add_button)
buttons_layout.addWidget(self.delete_button)

main_layout.addLayout(buttons_layout)
```

Менеджеры компоновки можно вкладывать друг в друга. Благодаря этому интерфейс формируется из простых логических областей и автоматически перестраивается при изменении размера окна.

7. Сигналы и слоты

Основой обработки событий в Qt является механизм сигналов и слотов. Сигнал сообщает о наступлении события, а слот представляет функцию или метод, который должен быть выполнен в ответ на это событие.

Например, сигнал clicked кнопки можно связать с методом add_task:

```
self.add_button.clicked.connect(self.add_task)
```

После такой связи при каждом нажатии кнопки PyQt6 автоматически вызывает метод self.add_task(). Один обработчик может быть связан с несколькими сигналами, а один сигнал может вызывать несколько обработчиков.

8. Основное окно приложения

Для простых окон можно использовать QWidget. Для главного окна настольного приложения удобно применять QMainWindow. Он поддерживает центральную область, меню, панели инструментов и строку состояния.

```
from PyQt6.QtWidgets import QMainWindow


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Менеджер задач")
```

В данной лабораторной работе класс используется как стандартный способ объединить окно, его элементы и обработчики событий. Подробно механизм классов и наследования рассматривается в последующих лабораторных работах.

9. Центральный виджет QMainWindow

QMainWindow имеет специальную структуру, поэтому менеджер компоновки не назначается непосредственно главному окну. Сначала создаётся центральный QWidget, который затем устанавливается с помощью setCentralWidget().

```
central_widget = QWidget()
self.setCentralWidget(central_widget)


layout = QVBoxLayout()
central_widget.setLayout(layout)
```

10. Обработка пользовательского ввода

Обработчик события обычно получает данные из элементов интерфейса, проверяет их и изменяет состояние приложения. Например, добавление задачи можно реализовать следующим образом:

```
def add_task(self):
    text = self.task_input.text().strip()

    if text:
        self.task_list.addItem(text)
        self.task_input.clear()
```

Метод strip() удаляет пробельные символы в начале и конце строки. Благодаря этому строка, состоящая только из пробелов, не должна считаться корректным названием задачи.

11. Работа с выбранным элементом списка

Для удаления или изменения конкретной задачи необходимо определить, какой элемент списка выбрал пользователь.

```
row = self.task_list.currentRow()

if row >= 0:
    self.task_list.takeItem(row)
```

Если ни один элемент не выбран, currentRow() возвращает отрицательное значение. Перед выполнением операции с выбранной строкой это состояние необходимо проверить.

12. Диалоговые окна QMessageBox

Для вывода уведомлений, предупреждений и запросов подтверждения используется QMessageBox. Диалоговое окно позволяет сообщить пользователю об ошибке или запросить решение перед выполнением важного действия.

```
QMessageBox.warning(
    self,
    "Ошибка",
    "Введите название задачи"
)
```

Для запроса подтверждения можно использовать QMessageBox.question(). Например, перед удалением выбранной задачи программа может предложить пользователю подтвердить действие.

```
answer = QMessageBox.question(
    self,
    "Удаление",
    "Удалить выбранную задачу?"
)

if answer == QMessageBox.StandardButton.Yes:
    ...
```

13. События различных виджетов

Сигналы существуют не только у кнопок. Например, QLineEdit формирует сигнал returnPressed при нажатии Enter. Его можно связать с тем же методом, что и кнопку добавления задачи.

```
self.task_input.returnPressed.connect(self.add_task)
```

Таким образом, один и тот же результат может быть доступен пользователю несколькими способами, а логика приложения при этом остаётся в одном обработчике.

14. Состояние графического интерфейса

Интерфейс должен отображать текущее состояние программы. Если список задач изменился, связанные с ним элементы также должны обновиться. Например, количество задач можно выводить в QLabel.

```
def update_counter(self):
    count = self.task_list.count()
    self.counter_label.setText(
        f"Количество задач: {count}"
    )
```

Метод update_counter() необходимо вызывать после операций, изменяющих список. Такой подход позволяет избежать несоответствия между данными программы и отображаемой пользователю информацией.

15. Разделение интерфейса и обработчиков

По мере роста приложения размещение всего кода внутри метода __init__() делает класс трудно читаемым. Создание интерфейса, подключение сигналов и обработку действий пользователя рекомендуется разделять между отдельными методами.

```
def __init__(self):
    super().__init__()
    self.init_ui()
    self.connect_signals()

def init_ui(self):
    ...

def connect_signals(self):
    ...

def add_task(self):
    ...

def delete_task(self):
    ...
```

16. Структура проекта

Практическая часть выполняется в одном проекте. В рамках данной лабораторной работы достаточно разделить точку входа и код основного окна на два Python-модуля.

```
task_manager/
    main.py
    main_window.py
    requirements.txt
    .gitignore
    .venv/
```

Каталог .venv содержит виртуальное окружение и не должен добавляться в Git. Файл requirements.txt фиксирует используемую версию PyQt6 и другие зависимости проекта.

ПРАКТИЧЕСКАЯ ЧАСТЬ

Все задания выполняются последовательно в одном проекте task_manager. После каждого этапа необходимо запускать приложение и проверять, что ранее реализованные функции продолжают работать.

Задание 1: Создание проекта и установка PyQt6

1. Создайте каталог task_manager и откройте его в Visual Studio Code.
2. Инициализируйте в каталоге Git-репозиторий.
3. Создайте виртуальное окружение .venv и активируйте его.
4. Установите PyQt6 командой python -m pip install PyQt6.
5. Создайте файл requirements.txt командой python -m pip freeze > requirements.txt.
6. Создайте .gitignore и добавьте в него .venv/, __pycache__/ и *.pyc.
7. Зафиксируйте первоначальную структуру проекта отдельным коммитом.

Задание 2: Создание первого окна

Создайте файлы main.py и main_window.py. В main.py реализуйте точку входа в приложение:

```
import sys

from PyQt6.QtWidgets import QApplication
from main_window import MainWindow

def main():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

В main_window.py создайте класс MainWindow, наследующий QMainWindow. Установите заголовок «Менеджер задач» и первоначальный размер окна 500 × 400 пикселей. Запустите приложение и убедитесь, что окно открывается.

Задание 3: Создание элементов интерфейса

Добавьте в главное окно следующие элементы:

• QLabel с текстом «Новая задача»;

• QLineEdit для ввода названия задачи;

• QPushButton «Добавить»;

• QListWidget для отображения задач;

• QPushButton «Удалить»;

• QPushButton «Очистить всё»;

• QLabel для отображения количества задач.

Для QLineEdit задайте подсказку «Название задачи». На данном этапе элементы должны только отображаться в окне.

Задание 4: Организация интерфейса

1. Создайте центральный QWidget и установите его в QMainWindow.
2. Используйте QVBoxLayout как основную вертикальную компоновку.
3. Поле ввода и кнопку «Добавить» расположите в одном QHBoxLayout.
4. Кнопки «Удалить» и «Очистить всё» расположите в другом QHBoxLayout.
5. Добавьте вложенные компоновки в основной layout.
6. Проверьте изменение интерфейса при растягивании окна.

Использование move() и абсолютных координат для основного размещения элементов в данной лабораторной работе не допускается.

Задание 5: Добавление задачи

Создайте метод add_task(). Свяжите его с сигналом clicked кнопки «Добавить». Метод должен:

1. получить строку из QLineEdit;
2. удалить начальные и конечные пробелы;
3. проверить, что строка не пустая;
4. добавить корректную строку в QListWidget;
5. очистить поле ввода после успешного добавления.

Задание 6: Обработка пустого ввода

Если пользователь пытается добавить пустую строку или строку, состоящую только из пробелов, задача не должна добавляться. Выведите QMessageBox.warning с понятным сообщением, например «Введите название задачи».

Задание 7: Добавление задачи клавишей Enter

Свяжите сигнал returnPressed поля QLineEdit с уже существующим методом add_task(). После изменения новая задача должна добавляться как нажатием кнопки «Добавить», так и нажатием Enter. Не создавайте отдельный обработчик с дублирующимся кодом.

Задание 8: Удаление выбранной задачи

Создайте метод delete_task() и свяжите его с кнопкой «Удалить». Метод должен определить выбранную строку QListWidget. Если строка не выбрана, необходимо вывести предупреждение через QMessageBox.warning.

Задание 9: Подтверждение удаления

Перед фактическим удалением выбранной задачи запросите подтверждение с помощью QMessageBox.question. Элемент должен удаляться только после выбора пользователем ответа Yes.

Задание 10: Очистка списка задач

Создайте метод clear_tasks(), вызываемый кнопкой «Очистить всё». Перед очисткой запросите подтверждение пользователя. Если список уже пуст, приложение не должно завершаться с ошибкой.

Задание 11: Счётчик задач

Создайте метод update_counter(), который отображает в QLabel количество элементов QListWidget в формате:

```
Количество задач: 3
```

Счётчик должен автоматически обновляться после успешного добавления задачи, удаления выбранной задачи и полной очистки списка.

Задание 12: Разделение кода

Организуйте класс MainWindow так, чтобы создание интерфейса, подключение сигналов и обработчики действий пользователя находились в отдельных методах. В классе должны присутствовать как минимум методы:

```
init_ui()
connect_signals()
add_task()
delete_task()
clear_tasks()
update_counter()
```

Метод __init__() не должен содержать всю логику программы в виде одного большого блока кода.

Задание 13: Самостоятельная работа

Доработайте созданный менеджер задач. Необходимо самостоятельно выбрать и реализовать не менее трёх дополнительных возможностей из списка:

• отметка выбранной задачи как выполненной;

• редактирование текста выбранной задачи;

• фильтрация отображаемых задач;

• выбор категории задачи с помощью QComboBox;

• выбор приоритета задачи;

• удаление только выполненных задач;

• отображение количества выполненных задач;

• изменение заголовка окна в зависимости от количества задач;

• использование строки состояния QMainWindow.

Дополнительные функции должны быть встроены в существующий проект, использовать сигналы и обработчики и не нарушать работу уже реализованных возможностей. Выбор необходимых виджетов и структура методов определяются студентом самостоятельно.

Требования к результату работы

После выполнения лабораторной работы студент должен уметь:

• объяснять отличие графического приложения от консольной программы;

• объяснять принцип событийно-ориентированной модели выполнения;

• создавать QApplication и главное окно PyQt6;

• использовать QLabel, QLineEdit, QPushButton, QListWidget и QMessageBox;

• использовать QVBoxLayout и QHBoxLayout для организации интерфейса;

• связывать сигналы виджетов с методами-обработчиками;

• получать и изменять состояние элементов интерфейса;

• обрабатывать некорректные действия пользователя;

• поддерживать согласованное состояние интерфейса после действий пользователя;

• разделять код точки входа, создания интерфейса и обработки событий;

• устанавливать PyQt6 в виртуальное окружение и фиксировать зависимость в requirements.txt.

Контрольные вопросы для самопроверки

1. Что такое графический пользовательский интерфейс?
2. Чем GUI-приложение отличается от консольной программы?
3. Что означает событийно-ориентированная модель программирования?
4. Для чего используется QApplication?
5. Для чего выполняется app.exec()?
6. Что такое виджет в PyQt6?
7. Чем QWidget отличается от QMainWindow?
8. Для чего в QMainWindow используется центральный виджет?
9. Для чего применяются менеджеры компоновки?
10. Чем QVBoxLayout отличается от QHBoxLayout?
11. Почему не рекомендуется размещать весь интерфейс с помощью абсолютных координат?
12. Что такое сигнал в PyQt6?
13. Что такое слот или обработчик события?
14. Для чего используется метод connect()?
15. Как получить текст, введённый в QLineEdit?
16. Как добавить элемент в QListWidget и определить выбранную строку?
17. Для чего используется QMessageBox?
18. Как обработать нажатие Enter в QLineEdit?
19. Почему состояние интерфейса необходимо обновлять после изменения данных?
20. Почему код создания интерфейса и обработчики событий рекомендуется разделять между отдельными методами?

Лабораторная работа №7: Качество и надёжность Python-кода: стиль, статический анализ, отладка и тестирование

Цель работы: изучить основные принципы написания читаемого, поддерживаемого и надёжного Python-кода; познакомиться с рекомендациями PEP 8; научиться применять автоматическое форматирование и статический анализ исходного кода с помощью Ruff, использовать средства отладки IDE, находить логические ошибки, создавать простые автоматизированные тесты с использованием assert и pytest, а также выполнять базовый рефакторинг программного кода.

В ходе выполнения лабораторной работы необходимо последовательно проанализировать и улучшить существующий Python-проект: выявить проблемы читаемости и оформления, выполнить автоматическое форматирование и статический анализ, найти и исправить логические ошибки с помощью отладчика, разработать набор тестов и убедиться, что после рефакторинга поведение программы остаётся корректным.

ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1. Качество программного кода

Работающая программа не обязательно является качественной. Код должен не только выдавать правильный результат, но и оставаться понятным для других разработчиков и самого автора спустя длительное время.

При оценке качества программного кода обычно обращают внимание на читаемость, понятность структуры, единообразие оформления, возможность изменения и расширения программы, простоту поиска ошибок и возможность автоматической проверки поведения.

Сравним два фрагмента, выполняющих одну и ту же операцию.

```
a = [10, 20, 30]
s = 0
for x in a:
    s += x
print(s / len(a))
```

Более понятный вариант может выглядеть следующим образом:

```
measurements = [10, 20, 30]

total = 0
for measurement in measurements:
    total += measurement

average = total / len(measurements)
print(average)
```

Вторая версия несколько длиннее, однако имена переменных отражают назначение данных и упрощают чтение программы.

2. PEP 8

PEP 8 — руководство по стилю оформления Python-кода. Оно содержит рекомендации, призванные сделать программы единообразными и читаемыми. PEP 8 описывает правила использования отступов и пробелов, расположения импортов, длины строк, пустых строк и именования программных объектов.

PEP 8 не определяет алгоритм работы программы и не гарантирует отсутствие ошибок. Его задача — сформировать единый стиль оформления исходного кода.

3. Отступы, пробелы и оформление выражений

В Python отступ является частью синтаксиса. Стандартно для одного уровня вложенности используются 1 TAB (четыре пробела).

```
if value > 0:
    print("Положительное число")
```

Не рекомендуется смешивать табуляцию и пробелы. В выражениях пробелы обычно используются вокруг бинарных операторов и после запятых.

```
result = first_value + second_value * 2
```

Оформление должно помогать визуально отделять логические части программы, а не усложнять их восприятие.

4. Именование переменных, функций и классов

Имена программных объектов должны отражать их назначение. Однобуквенные и слишком сокращённые имена допустимы только тогда, когда их смысл очевиден из контекста.

```
a = 10
b = 20
c = a + b
```

Более понятный вариант:

```
first_value = 10
second_value = 20
total = first_value + second_value
```

Для переменных и функций обычно применяется стиль snake_case:

```
measurement_count = 10


def calculate_average(numbers):
    ...
```

Для классов используется стиль PascalCase, например MeasurementAnalyzer.

5. Импорты и комментарии

Импорты обычно располагаются в начале файла. Независимые модули следует импортировать отдельными строками.

```
import json
import logging
from pathlib import Path
```

Комментарий должен пояснять неочевидное решение, причину ограничения или важную особенность алгоритма. Не следует добавлять комментарии, которые просто повторяют код.

```
# Первое измерение является служебным и не участвует в расчётах.
measurements = measurements[1:]
```

6. Автоматическое форматирование с помощью Ruff

Часть требований к оформлению может применяться автоматически. В данной лабораторной работе используется инструмент Ruff, который предоставляет средства форматирования и статического анализа Python-кода.

Установка Ruff в активированное виртуальное окружение:

```
python -m pip install ruff
```

Проверка форматирования без изменения файлов:

```
ruff format --check .
```

Автоматическое форматирование проекта:

```
ruff format .
```

Форматтер изменяет представление исходного кода, однако не исправляет алгоритмические и логические ошибки программы.

7. Статический анализ кода

Статический анализ выполняется без обычного запуска программы. Он анализирует исходный код и позволяет обнаружить часть потенциальных проблем: неиспользуемые импорты, некоторые ошибки оформления, отдельные подозрительные конструкции и другие нарушения.

Запуск Ruff для анализа проекта:

```
ruff check .
```

Часть проблем Ruff способен исправить автоматически:

```
ruff check . --fix
```

Автоматическое исправление следует использовать осознанно: после его выполнения необходимо просмотреть изменения и повторно проверить работу программы.

8. Виды ошибок в программах

Ошибки в программном коде можно условно разделить на синтаксические ошибки, исключения во время выполнения и логические ошибки.

8.1. Синтаксические ошибки

Синтаксическая ошибка нарушает правила языка и не позволяет интерпретатору корректно разобрать программу.

```
if value > 10
    print(value)
```

8.2. Ошибки времени выполнения

Программа может быть синтаксически правильной, но при определённых данных завершиться исключением.

```
result = 10 / 0
```

В данном случае возникнет ZeroDivisionError.

8.3. Логические ошибки

При логической ошибке программа может выполняться без исключений, но выдавать неправильный результат.

```
def calculate_average(numbers):
    return sum(numbers) / (len(numbers) + 1)
```

Такую ошибку линтер может не обнаружить, поскольку код синтаксически корректен. Для её поиска требуется анализ алгоритма, отладка или тестирование.

9. Отладка программы

Отладчик позволяет приостанавливать выполнение программы и наблюдать её состояние. Основными инструментами являются точки останова, пошаговое выполнение и просмотр значений переменных.

- Breakpoint — точка, в которой выполнение программы приостанавливается.
- Continue — продолжение выполнения до следующей точки останова.

• Step Over — выполнение текущей строки без перехода внутрь вызываемой функции.

• Step Into — переход внутрь вызываемой функции.

• Step Out — завершение текущей функции и возврат к месту её вызова.

Названия команд и сочетания клавиш могут различаться в зависимости от используемой IDE, однако принцип работы остаётся одинаковым.

10. Поиск логической ошибки с помощью отладчика

Рассмотрим функцию подсчёта положительных чисел:

```
def count_positive(numbers):
    count = 0

    for number in numbers:
        if number >= 0:
            count += 1

    return count
```

Если по условию задачи ноль не считается положительным числом, выражение number >= 0 является логической ошибкой. С помощью пошагового выполнения можно проследить значение number и определить, при каком значении счётчик изменяется неверно.

11. Проверки с помощью assert

Оператор assert позволяет проверить, что некоторое условие является истинным. Если условие выполняется, программа продолжает работу. Если условие ложно, возникает AssertionError.

```
result = calculate_average([10, 20, 30])
assert result == 20
```

Можно добавить сообщение:

```
assert result == 20, "Среднее значение рассчитано неправильно"
```

Assert удобен для простых проверок во время разработки, однако для систематического автоматизированного тестирования обычно используется специальный тестовый фреймворк.

12. Автоматизированное тестирование

Тест — программный код, проверяющий ожидаемое поведение другого кода. Тест вызывает проверяемую функцию с заранее выбранными входными данными и сравнивает фактический результат с ожидаемым.

```
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5
```

Один тест редко покрывает все возможные случаи. Для функции следует проверять обычные значения, граничные случаи и ошибочные входные данные.

13. pytest

В лабораторной работе для автоматизированного тестирования используется pytest. Он позволяет создавать тесты в виде обычных Python-функций и применять стандартный assert.

Установка:

```
python -m pip install pytest
```

Пример файла tests/test_calculations.py:

```
from calculations import calculate_average

def test_calculate_average():
    result = calculate_average([10, 20, 30])
    assert result == 20
```

Запуск тестов из корня проекта:

```
pytest
```

pytest автоматически обнаруживает тестовые файлы и функции в соответствии с принятой системой именования.

14. Несколько тестовых случаев

Одну функцию следует проверять на нескольких наборах входных данных. Например:

```
def test_average_positive():
    assert calculate_average([10, 20, 30]) == 20

def test_average_single_value():
    assert calculate_average([5]) == 5

def test_average_negative():
    assert calculate_average([-10, 0, 10]) == 0
```

Такой набор лучше описывает ожидаемое поведение функции и повышает вероятность обнаружения ошибки после изменения кода.

15. Тестирование исключений

Если функция должна отклонять некорректные входные данные, тест может проверить возникновение ожидаемого исключения.

```
import pytest

def test_average_empty_list():
    with pytest.raises(ValueError):
        calculate_average([])
```

16. Регрессия

Регрессией называют ситуацию, когда после изменения программы ранее работавшая функциональность начинает работать неправильно. Автоматические тесты позволяют быстро повторно проверить уже известное поведение после добавления новых возможностей или изменения внутренней структуры кода.

17. Рефакторинг

Рефакторинг — изменение внутренней структуры кода без намеренного изменения его внешнего поведения. Цель рефакторинга — сделать программу понятнее, проще для сопровождения и расширения.

К базовым приёмам рефакторинга относятся:

- переименование переменных и функций;
- удаление дублирующегося кода;
- разбиение большой функции на несколько небольших;
- выделение повторяющегося кода в отдельную функцию;
- упрощение условий;
- удаление неиспользуемого кода.

Перед рефакторингом полезно убедиться, что тесты проходят. После изменения структуры кода тесты необходимо запустить повторно. Если они по-прежнему проходят, это является дополнительным подтверждением того, что внешнее поведение программы сохранено.

ПРАКТИЧЕСКАЯ ЧАСТЬ

Вся практическая часть выполняется в одном проекте. В начале студент получает небольшой модуль с намеренно заложенными проблемами оформления, плохими именами и логическими ошибками. В ходе лабораторной работы проект последовательно приводится в качественное состояние.

Задание 1: Создание проекта

Создайте каталог code_quality_lab и следующую структуру проекта:

```
code_quality_lab/
    main.py
    calculations.py
    requirements.txt
    tests/
        test_calculations.py
```

Создайте и активируйте виртуальное окружение. Установите Ruff и pytest:

```
python -m pip install ruff pytest
```

Сформируйте файл зависимостей:

```
python -m pip freeze > requirements.txt
```

Задание 2: Исходный код с проблемами

Поместите в calculations.py следующий код без предварительного исправления:

```
import math
import json


def calc(a):
  s=0
  for x in a:
      s=s+x
  return s/(len(a)+1)


def mx(a):
    m=0
    for x in a:
        if x>m:
            m=x
    return m


def countpos(a):
    c=0
    for x in a:
        if x>=0:
            c=c+1
    return c
```

До запуска автоматических инструментов самостоятельно изучите код и зафиксируйте в отчёте не менее пяти обнаруженных проблем. Для каждой проблемы укажите фрагмент кода и предлагаемое исправление.

Задание 3: Проверка и автоматическое форматирование

Выполните проверку форматирования:

```
ruff format --check .
```

Зафиксируйте результат. После этого выполните:

```
ruff format .
```

Повторно откройте calculations.py и сравните исходный код с отформатированным. Укажите в отчёте, какие изменения Ruff выполнил автоматически.

Задание 4: Статический анализ

Запустите статический анализ проекта:

```
ruff check .
```

Изучите обнаруженные проблемы. Исправьте их вручную либо примените автоматическое исправление для тех случаев, где это возможно:

```
ruff check . --fix
```

Повторно выполните ruff check . и добейтесь отсутствия обнаруживаемых Ruff проблем.

Задание 5: Поиск логической ошибки среднего значения

В main.py вызовите исходную функцию calc для списка [10, 20, 30]. Ожидаемое среднее значение равно 20. Если фактический результат отличается, не исправляйте функцию сразу.

Установите точку останова в функции и с помощью отладчика определите значения суммы элементов, длины списка и выражения, используемого при делении. Объясните причину ошибки и только после этого исправьте функцию.

Задание 6: Поиск ошибки максимального значения

Проверьте функцию mx на наборе:

```
[-10, -5, -20]
```

Определите ожидаемый и фактический результаты. Используя пошаговую отладку, проследите изменение переменной, содержащей текущее максимальное значение. Найдите причину ошибки и исправьте алгоритм.

Задание 7: Анализ подсчёта положительных чисел

Проверьте функцию countpos на наборе:

```
[-1, 0, 1]
```

Сопоставьте результат с математическим определением положительного числа. При необходимости исправьте условие и зафиксируйте причину изменения.

Задание 8: Рефакторинг имён

Переименуйте функции и переменные так, чтобы их назначение было понятно без дополнительного пояснения. В результате должны появиться функции с понятными именами, например calculate_average, find_max и count_positive. Однобуквенные имена внутренних переменных также замените на содержательные там, где это улучшает читаемость.

Задание 9: Проверки с помощью assert

Добавьте простые проверки исправленных функций:

```
assert calculate_average([10, 20, 30]) == 20
assert find_max([-10, -5, -20]) == -5
assert count_positive([-1, 0, 1]) == 1
```

Запустите программу и убедитесь, что все проверки выполняются успешно. Затем временно измените одно ожидаемое значение и изучите сообщение AssertionError. После этого верните правильную проверку.

Задание 10: Создание тестов pytest

В файле tests/test_calculations.py создайте отдельные тестовые функции для calculate_average, find_max и count_positive. Для каждой проверяемой функции используйте минимум два различных набора входных данных.

Запустите тесты:

```
pytest
```

Зафиксируйте итоговый вывод pytest в отчёте.

Задание 11: Проверка ошибочного изменения

Временно верните логическую ошибку в одну из функций, например измените формулу среднего значения. Повторно выполните pytest и

определите, какой тест обнаружил проблему, какой результат ожидался и какой был фактически получен.

После анализа верните корректную реализацию и убедитесь, что все тесты снова проходят.

Задание 12: Обработка пустого списка

Измените функцию calculate_average так, чтобы при передаче пустого списка она создавала исключение ValueError. Добавьте соответствующий тест:

```
import pytest


def test_average_empty_list():
    with pytest.raises(ValueError):
        calculate_average([])
```

Убедитесь, что тест проходит.

Задание 13: Финальный рефакторинг

Проанализируйте проект повторно. Улучшите структуру и читаемость кода, удалите ненужные импорты и дублирование, упростите условия, если это возможно, и убедитесь, что функции имеют понятные названия и назначение.

После завершения рефакторинга выполните последовательно:

```
ruff format .
ruff check .
pytest
```

Финальный проект должен успешно проходить статический анализ и все разработанные тесты.

Задание 14: Самостоятельная работа

Добавьте в calculations.py следующую функцию в исходном виде:

```
def analize(a):
    if len(a)==0:
        return None
    s=0
    mx=0
    mn=0
    for x in a:
        s+=x
        if x>mx:
            mx=x
        if x<mn:
            mn=x
    return mn,mx,s/len(a)
```

Необходимо самостоятельно выполнить полный цикл улучшения этой функции:

1. проанализировать качество исходного кода и перечислить обнаруженные проблемы;
2. найти логические ошибки;
3. переименовать функцию и переменные;
4. выполнить форматирование Ruff;
5. добиться успешного прохождения ruff check .;
6. исправить алгоритм;
7. написать не менее пяти pytest-тестов для различных входных данных;
8. предусмотреть корректную обработку пустого списка;
9. выполнить итоговый рефакторинг;
10. повторно запустить Ruff и pytest и зафиксировать успешный результат.

Готовое решение функции в методических указаниях не приводится. Студент должен самостоятельно определить итоговую структуру реализации.

Требования к результату работы

После выполнения лабораторной работы студент должен уметь:

- объяснять назначение рекомендаций PEP 8;

• отличать читаемый и поддерживаемый код от просто работающего;

• применять основные правила именования и оформления Python-кода;

• использовать Ruff для автоматического форматирования и статического анализа;

• объяснять ограничения статического анализа;

• различать синтаксические, runtime- и логические ошибки;

• использовать точки останова и пошаговое выполнение в IDE;

• анализировать значения переменных во время выполнения программы;

• использовать assert для простых проверок;

• создавать и запускать тесты pytest;

• проверять возникновение ожидаемых исключений;

• объяснять понятия регрессии и рефакторинга;

• выполнять базовый рефакторинг с повторной проверкой программы тестами.

Контрольные вопросы для самопроверки

1. Что понимается под качеством программного кода?
2. Для чего используется PEP 8?
3. Какие правила именования обычно применяются к переменным, функциям и классам Python?
4. Чем автоматическое форматирование отличается от статического анализа?
5. Для чего используется команда ruff format?
6. Для чего используется команда ruff check?
7. Какие проблемы может обнаружить статический анализатор?
8. Почему линтер не может гарантировать правильность алгоритма?
9. Чем синтаксическая ошибка отличается от логической?
10. Что такое breakpoint и для чего он используется?
11. Для чего применяется пошаговое выполнение программы?
12. Чем Step Over отличается от Step Into?
13. Для чего используется assert?

14. Что такое автоматизированный тест?

15. Для чего используется pytest?

16. Почему одной функции обычно требуется несколько тестовых случаев?

17. Как с помощью pytest проверить возникновение ожидаемого исключения?

18. Что такое регрессия?

19. Что такое рефакторинг?

20. Почему после рефакторинга необходимо повторно запускать статический анализ и тесты?

Лабораторная работа №8: Основы операционных систем Linux

Цель работы: изучить основные принципы организации операционных систем семейства Linux, познакомиться с архитектурой Linux, назначением ядра и дистрибутивов, структурой файловой системы, командной оболочкой, базовыми командами, системой пользователей и прав доступа, средствами управления программными пакетами и ролью Linux в современной ИТ-инфраструктуре.

В ходе выполнения лабораторной работы необходимо научиться ориентироваться в файловой системе Linux, выполнять основные операции с файлами и каталогами из командной оболочки, использовать перенаправление и конвейеры, выполнять поиск, анализировать права доступа, работать с процессами и пакетами, а также понимать место Linux в современной инфраструктуре разработки и эксплуатации программного обеспечения.

ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1. Операционная система и Linux

Операционная система — комплекс программных средств, обеспечивающий взаимодействие прикладных программ и пользователя с аппаратным обеспечением компьютера. Операционная система управляет процессами, оперативной памятью, устройствами, файловыми системами, пользователями и другими ресурсами.

Упрощённо взаимодействие компонентов можно представить следующим образом:

```
Пользователь / приложение
        ↓
Операционная система
        ↓
Аппаратное обеспечение
```

Linux в строгом смысле представляет собой ядро операционной системы. Ядро отвечает за управление процессами, памятью, устройствами,

файловыми системами, сетью и предоставляет приложениям системные интерфейсы для работы с ресурсами компьютера.

Дистрибутив Linux объединяет ядро, системные утилиты, библиотеки, менеджер пакетов, командную оболочку и дополнительное программное обеспечение. К распространённым дистрибутивам относятся Ubuntu, Debian, Fedora, Arch Linux и Rocky Linux. В рамках лабораторной работы используется Ubuntu либо совместимая Linux-среда.

Linux широко применяется на серверах, в облачной инфраструктуре, контейнерных средах, системах CI/CD, сетевом оборудовании, встраиваемых устройствах и вычислительных системах.

2. Командная оболочка и терминал

Терминал предоставляет пользователю интерфейс для ввода команд, а командная оболочка интерпретирует введённые команды и запускает соответствующие программы. Одной из наиболее распространённых оболочек является Bash.

Приглашение командной строки может выглядеть следующим образом:

```
student@ubuntu:~/project$
```

Здесь student — имя пользователя, ubuntu — имя компьютера, ~/project — текущий каталог, а символ $ обычно обозначает работу от имени обычного пользователя. Символ # часто используется в приглашении оболочки пользователя root.

3. Файловая система Linux

В Linux используется единое дерево файловой системы, начинающееся с корневого каталога /. В отличие от Windows, отдельные диски не представлены независимыми деревьями C:, D: и т. п., а подключаются к общему дереву каталогов.

```
/
├── home
├── etc
├── var
├── usr
├── tmp
├── dev
└── proc
```

Основные каталоги, с которыми полезно познакомиться на начальном этапе:

- / — корневой каталог файловой системы;
- /home — домашние каталоги пользователей;
- /etc — системные конфигурационные файлы;
- /var — изменяемые данные, в том числе журналы работы программ;
- /tmp — временные файлы;
- /usr — программы, библиотеки и другие системные ресурсы;
- /dev — представления устройств;
- /proc — виртуальная файловая система с информацией о процессах и состоянии ядра.

Путь может быть абсолютным или относительным. Абсолютный путь начинается от корня файловой системы:

```
/home/student/project/main.py
```

Относительный путь задаётся относительно текущего рабочего каталога:

```
project/main.py
```

Специальные обозначения: . — текущий каталог, .. — родительский каталог, ~ — домашний каталог текущего пользователя.

4. Навигация и работа с файлами

Для определения текущего каталога используется команда:

```
pwd
```

Для просмотра содержимого каталога:

```
ls
```

Полезные варианты:

```
ls -l
ls -a
ls -la
```

Переход между каталогами выполняется командой cd:

```
cd /home/student
cd ..
cd ~
```

Создание каталогов и файлов:

```
mkdir project
touch file.txt
mkdir -p project/data/input
```

Копирование, перемещение, переименование и удаление:

```
cp source.txt copy.txt
mv old.txt new.txt
rm file.txt
cp -r source_dir destination_dir
rm -r directory
```

Команда rm удаляет файлы без помещения в корзину. Параметры рекурсивного и принудительного удаления следует использовать только при понимании последствий операции.

Для просмотра содержимого текстовых файлов используются команды:

```
cat file.txt
less file.txt
head file.txt
tail file.txt
```

Команда tail -f удобна для наблюдения за дописываемым журналом работы программы в реальном времени.

5. Перенаправление и конвейеры

Стандартный вывод команды можно перенаправить в файл. Оператор > создаёт файл или перезаписывает существующее содержимое:

```
echo "Hello" > file.txt
```

Оператор >> добавляет данные в конец файла:

```
echo "Second line" >> file.txt
```

Конвейер | передаёт стандартный вывод одной команды на стандартный ввод другой команды:

```
ls -l | less
```

Конвейеры позволяют объединять небольшие утилиты в последовательности обработки данных.

6. Поиск файлов и данных

Команда find предназначена для поиска объектов файловой системы. Например, найти все Python-файлы в текущем каталоге и его подкаталогах:

```
find . -name "*.py"
```

Команда grep выполняет поиск строк по текстовому шаблону:

```
grep "ERROR" app.log
```

Для рекурсивного поиска по каталогу можно использовать:

```
grep -r "TODO" .
```

Команды find и grep решают разные задачи: find ищет файлы и каталоги по их свойствам, grep — данные внутри текстового содержимого.

7. Пользователи и права доступа

Linux является многопользовательской системой. Файлы и процессы связаны с пользователями и группами, а доступ к объектам ограничивается системой прав.

Получить имя текущего пользователя и его идентификаторы можно командами:

```
whoami
id
```

Пользователь root обладает расширенными административными правами. Команда sudo позволяет авторизованному пользователю выполнить отдельную команду с повышенными привилегиями:

```
sudo apt update
```

Повышенные права не следует использовать без необходимости. Ошибка Permission denied не означает автоматически, что команду следует повторить через sudo.

Права файла отображаются, например, следующим образом:

```
-rwxr-xr--
```

Для каждого объекта задаются права владельца (owner), группы (group) и остальных пользователей (others). Основные обозначения:

- r — чтение (read);
- w — запись (write);
- x — выполнение (execute).

Для просмотра прав используется ls -l. Для изменения — chmod:

```
chmod u+x script.sh
chmod g-w file.txt
```

Права также можно задавать числом: r = 4, w = 2, x = 1. Например:

```
chmod 755 script.sh
```

В режиме 755 владелец получает rwx, а группа и остальные пользователи — r-x.

8. Исполняемые файлы и Bash-скрипты

Bash-скрипт представляет собой текстовый файл с последовательностью команд оболочки. Первая строка может содержать shebang, указывающий интерпретатор:

```
#!/bin/bash


echo "Hello Linux"
```

Чтобы запустить файл напрямую, ему необходимо право выполнения:

```
chmod +x script.sh
./script.sh
```

Префикс ./ указывает оболочке, что программа находится в текущем каталоге.

9. Процессы

Процесс — выполняющийся экземпляр программы. Каждый процесс имеет идентификатор PID, владельца и текущее состояние.

Для просмотра процессов используются команды:

```
ps
ps aux
top
```

Завершение процесса можно выполнить командой kill, передав идентификатор PID:

```
kill PID
```

Команда kill отправляет процессу сигнал. Подробная работа с сигналами выходит за рамки данной лабораторной работы.

10. Управление программными пакетами

В Linux программное обеспечение часто устанавливается из репозиториев дистрибутива при помощи менеджера пакетов. Пакет содержит программу или библиотеку вместе с необходимыми метаданными.

В Ubuntu используется менеджер пакетов APT. Обновление информации о доступных пакетах:

```
sudo apt update
```

Установка и удаление пакета:

```
sudo apt install <package>
sudo apt remove <package>
```

Менеджер пакетов получает программу из настроенных репозиториев и автоматически учитывает необходимые системные зависимости.

11. Системные пакеты и Python-пакеты: apt и pip

Важно различать менеджер системных пакетов и менеджер Python-пакетов. APT управляет пакетами операционной системы и программами,

поставляемыми репозиториями дистрибутива. pip устанавливает пакеты из экосистемы Python в выбранное Python-окружение.

```
sudo apt install tree
python -m pip install pytest
```

Эти инструменты работают на разных уровнях и не являются взаимозаменяемыми.

12. Получение информации о системе

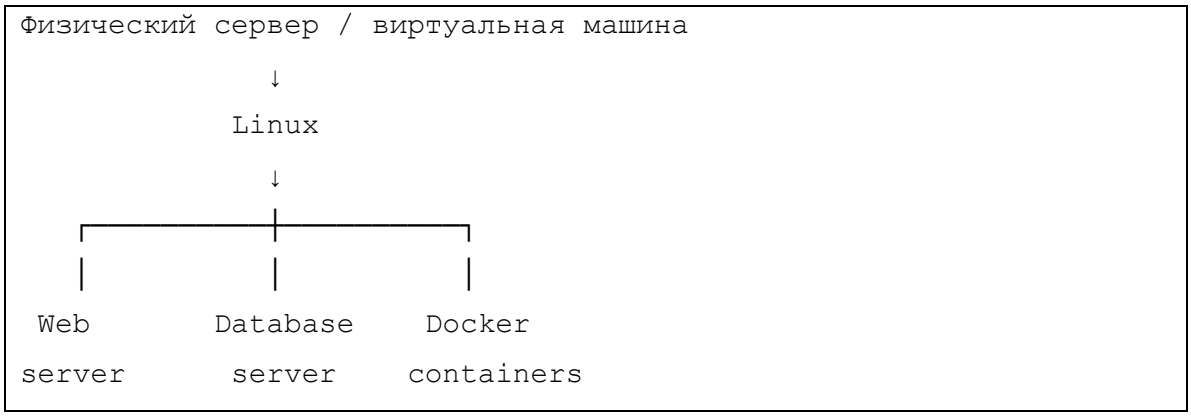
Для диагностики системы полезно уметь получать базовую информацию из командной строки.

```
uname -a
uname -r
hostname
cat /etc/os-release
df -h
free -h
```

uname отображает информацию о ядре, hostname — имя компьютера, /etc/os-release содержит сведения о дистрибутиве, df -h показывает использование дискового пространства, free -h — состояние оперативной памяти.

13. Linux в современной ИТ-инфраструктуре

Linux широко используется как серверная платформа. На нём запускаются веб-серверы, системы управления базами данных, средства автоматизации, контейнерные платформы и множество облачных сервисов.



Для разработчика знание Linux важно не только при непосредственной работе на Linux-компьютере. Linux-среды регулярно встречаются в WSL, Docker-контейнерах, виртуальных машинах, CI/CD и облачных сервисах.

ПРАКТИЧЕСКАЯ ЧАСТЬ

Практическая часть выполняется в Ubuntu или другой совместимой Linux-среде. Если на компьютере используется Windows, допускается работа в Ubuntu через WSL2. Основным объектом работы является один каталог ~/linux_lab, который последовательно развивается на протяжении всей лабораторной работы.

Задание 1: Получение информации о системе

Выполните команды:

```
whoami
hostname
uname -r
cat /etc/os-release
pwd
```

Определите текущего пользователя, имя компьютера, версию ядра, используемый дистрибутив и текущий рабочий каталог. Зафиксируйте команды и результаты в отчёте.

Задание 2: Навигация по файловой системе

Перейдите в домашний каталог и изучите его содержимое:

```
cd ~
pwd
ls
ls -la
```

Выполните переход в несколько каталогов, используя абсолютный и относительный пути, а также обозначения . , .. и ~. В отчёте объясните различия между этими способами задания пути.

Задание 3: Создание структуры лабораторного проекта

В домашнем каталоге создайте следующую структуру:

```
~/linux_lab/
├── data/
├── logs/
├── scripts/
└── README.txt
```

Команды для создания структуры подберите самостоятельно. Продемонстрируйте результат через ls -la или установленную позднее команду tree.

Задание 4: Работа с текстовым файлом

С помощью команды echo запишите в README.txt строку:

```
Linux laboratory project
```

Добавьте вторую строку без удаления первой. Просмотрите файл при помощи cat и less. В отчёте покажите отличие операторов > и >>.

Задание 5: Копирование, перемещение и удаление

Внутри ~/linux_lab создайте несколько текстовых файлов. Выполните копирование файла, его переименование, перемещение между каталогами data, logs и scripts, а затем удалите ненужную копию. После каждой операции проверяйте результат командой ls.

Задание 6: Создание и анализ журнала

Создайте файл logs/app.log со следующим содержимым:

```
INFO Application started
ERROR Cannot open configuration
INFO Application stopped
ERROR Connection failed
```

Найдите все строки, содержащие ERROR, с помощью grep. Затем самостоятельно составьте конвейер, который выводит количество найденных строк с ошибками. Для подсчёта строк можно использовать команду wc.

Задание 7: Поиск файлов

Создайте внутри проекта несколько файлов с расширениями .py, .txt и .log. Используя find, по отдельности найдите все файлы каждого из этих типов. Выполните поиск из корня каталога ~/linux_lab.

Задание 8: Создание исполняемого Bash-скрипта

Создайте файл scripts/hello.sh:

```
#!/bin/bash


echo "Hello from Linux"
```

Сначала попытайтесь выполнить файл напрямую:

```
./scripts/hello.sh
```

Если выполнение запрещено правами файла, изучите сообщение оболочки. Затем добавьте владельцу право выполнения:

```
chmod u+x scripts/hello.sh
```

Повторите запуск и зафиксируйте результат.

Задание 9: Анализ и изменение прав доступа

Просмотрите права файла:

```
ls -l scripts/hello.sh
```

Расшифруйте права владельца, группы и остальных пользователей. Затем установите режим:

```
chmod 754 scripts/hello.sh
```

Объясните, какие операции разрешены каждой категории пользователей после изменения.

Задание 10: Пользователь и группы

Выполните:

```
whoami
id
```

Определите UID, основной GID и группы, в которые входит текущий пользователь.

Задание 11: Работа с процессами

Запустите команду sleep в фоновом режиме:

```
sleep 300 &
```

Найдите созданный процесс через ps или ps aux, определите его PID и завершите процесс командой kill. После этого убедитесь, что процесс больше не выполняется.

Задание 12: Состояние системы

Выполните:

```
df -h
free -h
```

Определите общий и доступный объём дискового пространства и оперативной памяти. В отчёте кратко объясните назначение обеих команд.

Задание 13: Работа с пакетами

Проверьте наличие программы tree:

```
tree --version
```

Если программа отсутствует, установите её средствами APT:

```
sudo apt update
sudo apt install tree
```

После установки выведите структуру лабораторного проекта:

```
tree ~/linux_lab
```

В отчёте укажите, что выполняют команды apt update и apt install.

Задание 14: Сравнение apt и pip

На основе данной лабораторной и предыдущих лабораторных работ объясните различие между apt и pip. Укажите, какой инструмент использовался для установки системной программы tree и какой используется для установки библиотек Python в виртуальное окружение.

Задание 15: Самостоятельная работа

Создайте в домашнем каталоге проект:

```
~/server_project/
├── config/
├── data/
├── logs/
├── scripts/
└── backup/
```

Самостоятельно подберите команды Linux и выполните следующие действия:

1. создайте указанную структуру каталогов;

2. создайте конфигурационный текстовый файл внутри config;

3. создайте журнал с несколькими строками INFO, WARNING и ERROR;

4. найдите все строки ERROR;

5. определите количество строк ERROR с использованием конвейера;

6. создайте Bash-скрипт в каталоге scripts;

7. сделайте скрипт исполняемым и запустите его;

8. скопируйте конфигурационный файл в каталог backup;

9. найдите все файлы с расширением .log внутри проекта;

10. выведите итоговую структуру каталогов;

11. определите текущие права Bash-скрипта;

12. измените права так, чтобы владелец мог читать, изменять и выполнять файл, а группа и остальные пользователи — только читать и выполнять его.

Последовательность команд студент определяет самостоятельно. В отчёте необходимо показать исходное состояние, используемые команды и итоговый результат.

Требования к результату работы

После выполнения лабораторной работы студент должен уметь:

- объяснять назначение операционной системы, ядра Linux и дистрибутива;
- ориентироваться в структуре файловой системы Linux;
- использовать абсолютные и относительные пути;
- выполнять навигацию и основные операции с файлами и каталогами;
- использовать перенаправление стандартного вывода и конвейеры;
- выполнять поиск файлов и текстовых данных с помощью find и grep;
- объяснять назначение пользователей, групп и sudo;
- читать и изменять права доступа к файлам;
- создавать и запускать простые Bash-скрипты;

- просматривать и завершать процессы;
- получать базовую информацию о состоянии Linux-системы;
- устанавливать программные пакеты средствами APT;
- объяснять различие между apt и pip;
- понимать роль Linux в серверной, контейнерной и облачной инфраструктуре.

Контрольные вопросы для самопроверки

1. Что такое операционная система и какие основные задачи она выполняет?
2. Что такое ядро Linux?
3. Чем ядро Linux отличается от дистрибутива?
4. Назовите несколько распространённых дистрибутивов Linux.
5. Что такое командная оболочка и чем она отличается от терминала?
6. Что означает корневой каталог /?
7. Для чего используется каталог /home?
8. Чем абсолютный путь отличается от относительного?
9. Что обозначают . , .. и ~ в путях Linux?
10. Для чего используются команды pwd, ls и cd?
11. Чем отличаются операторы > и >>?
12. Что такое конвейер | и для чего он используется?
13. Чем отличаются команды find и grep?
14. Что означают права r, w и x?
15. На какие категории пользователей разделяются права доступа к файлу?
16. Для чего используется команда chmod?
17. Что означает режим доступа 755?
18. Для чего используется sudo и почему его не следует применять без необходимости?
19. Что такое процесс и что представляет собой PID?
20. Для чего используется менеджер пакетов apt?
21. Чем apt отличается от pip?

22. Почему Linux широко используется в серверной, контейнерной и облачной инфраструктуре?